

CollabQA: Colaboração Humano–IA na Qualidade de Software em Módulos de Autenticação

Diogo Pedro^{1*} and Guilherme Vicente¹

¹Departamento de Informática, Universidade da Beira Interior, Covilhã, Portugal.

*Corresponding author(s). E-mail(s): diogo.pedro@ubi.pt;
Contributing authors: guilherme.s.vicente@ubi.pt;

Abstract

A utilização de ferramentas de inteligência artificial (IA) em atividades de engenharia de software tem vindo a aumentar, especialmente em tarefas como revisão de código, geração de testes e apoio à depuração. Apesar do seu potencial para melhorar a produtividade e apoiar a qualidade de software, estas ferramentas podem produzir sugestões plausíveis que, se forem aceites sem validação humana adequada, podem introduzir ou deixar passar defeitos relevantes. Este projeto apresenta o CollabQA, um protótipo desenvolvido para estudar a colaboração Humano–IA em atividades de qualidade de software, usando como caso de estudo um módulo de autenticação. O projeto dá continuidade ao seminário sobre confiança, accountability e responsabilidade em engenharia de software apoiada por IA, operacionalizando os seus principais conceitos através de revisão de código assistida por IA, validação humana, sugestão de testes, registo de auditoria e comparação experimental. O caso de estudo inclui um defeito controlado relacionado com a validação de sessões expiradas. Inicialmente, a suíte de testes passava com sucesso, mas não detetava esse problema. Após a análise assistida por IA e a validação humana, foi sugerido um novo teste que falhou na implementação original, confirmando a existência do defeito. Depois da correção da função de validação de sessão, todos os testes passaram e a cobertura final atingiu 93%. Os resultados sugerem que a colaboração Humano–IA oferece um melhor equilíbrio do que a utilização isolada do humano ou da IA, ao combinar análise automatizada, julgamento humano, rastreabilidade e responsabilidade explícita.

Keywords: Qualidade de Software, Colaboração Humano–IA, Revisão de Código, Automação de Testes, Accountability, Autenticação

1 Introdução

A utilização de ferramentas de inteligência artificial (IA) em atividades de engenharia de software tem vindo a alterar a forma como o código é produzido, revisto, testado e mantido, especialmente com o crescimento de ferramentas de geração e revisão de código apoiadas por IA [1]. Ferramentas baseadas em IA, em particular modelos de linguagem de grande dimensão, conseguem apoiar tarefas como geração de código, revisão automática, criação de testes, explicação de erros e apoio à depuração. Estas capacidades podem aumentar a produtividade das equipas e contribuir para processos de qualidade de software mais rápidos e abrangentes.

Apesar destas vantagens, a integração de IA no desenvolvimento de software também introduz riscos relevantes. Ao contrário de ferramentas tradicionais, que executam operações geralmente previsíveis e diretamente controladas pelo utilizador, os sistemas baseados em IA produzem respostas probabilísticas. Essas respostas podem parecer corretas, bem estruturadas e convincentes, mas ainda assim conter erros, omitir casos críticos ou não respeitar completamente o contexto do projeto. Este problema torna-se particularmente importante em módulos sensíveis, como autenticação, autorização, gestão de sessões e controlo de permissões.

Neste contexto, a colaboração entre humanos e IA deve ser entendida como um processo controlado, no qual a IA apoia a análise, mas não substitui a decisão humana. Uma sugestão produzida por IA deve ser tratada como uma hipótese de trabalho que necessita de validação, testes e, quando aplicável, revisão por parte de um programador ou tester. Assim, a qualidade do resultado final não depende apenas da capacidade da ferramenta, mas também da forma como as sugestões são avaliadas, justificadas, documentadas e integradas no processo de desenvolvimento.

Este projeto apresenta o CollabQA, um protótipo desenvolvido no âmbito da unidade curricular de Qualidade de Software, com o objetivo de estudar a colaboração Humano-IA em atividades de revisão de código, sugestão de testes e análise de defeitos. O projeto utiliza como caso de estudo um pequeno módulo de autenticação com uma falha controlada na validação de sessões expiradas. Inicialmente, os testes existentes passavam com sucesso, criando uma falsa sensação de qualidade. No entanto, esses testes não verificavam se uma sessão já expirada continuava a ser aceite pelo sistema.

A principal contribuição do projeto consiste em transformar o caso de estudo discutido no seminário numa solução prática. O CollabQA permite que a IA identifique riscos no código, que o humano aceite, rejeite ou aceite parcialmente essas sugestões, e que o sistema registre as decisões tomadas num audit log. Além disso, o protótipo sugere testes associados aos riscos identificados e permite comparar três modos de avaliação: humano sozinho, IA sozinha e colaboração Humano-IA.

Os resultados obtidos mostram que a colaboração Humano-IA foi o modo mais equilibrado. A IA identificou três riscos relevantes, o humano validou essas sugestões, e o sistema registou três decisões auditáveis. A sugestão mais crítica originou um novo teste para sessões expiradas, que inicialmente falhou e confirmou a existência do defeito. Após a correção da função de validação de sessão, todos os testes passaram e a cobertura final atingiu 93%. Desta forma, o projeto demonstra que a IA pode apoiar a qualidade de software, desde que a decisão final permaneça humana, justificada e rastreável.

Adicionalmente, o projeto foi alinhado com as questões centrais do enunciado do Project #1, nomeadamente a comparação entre modelos *Human-in-the-Loop* (HITL) e *Human-on-the-Loop* (HOTL), a análise de riscos de *over-reliance* e *under-verification*, e a reflexão sobre a relação entre confiança e precisão das decisões. Assim, para além dos resultados técnicos, o trabalho inclui uma análise sobre comportamento de verificação, decisões aceites ou parcialmente aceites, erros detetados e limitações associadas à ausência de medição formal de tempo de decisão.

2 Enquadramento e Relação com o Seminário

A qualidade de software deve ser entendida como um conceito multidimensional, envolvendo atributos como adequação funcional, fiabilidade, segurança, manutenibilidade, usabilidade e portabilidade [2, 3].

Este projeto foi desenvolvido como continuação prática do seminário intitulado *Trust, Accountability, and Responsibility in AI-Supported Software Engineering*. No seminário, foi analisado o impacto da inteligência artificial (IA) em processos de engenharia de software, com especial atenção às questões de confiança, accountability e responsabilidade. A ideia central discutida foi que as ferramentas de IA podem apoiar atividades de desenvolvimento, revisão, teste e manutenção de software, mas não devem ser tratadas como substitutas autónomas do julgamento humano.

A relação entre o seminário e o presente projeto é direta. Enquanto o seminário apresentou uma reflexão conceptual sobre os riscos e oportunidades da utilização de IA na engenharia de software, este projeto transforma esses conceitos numa solução prática. O CollabQA foi criado para operacionalizar, num caso de estudo concreto, os princípios de confiança calibrada, validação humana, rastreabilidade e responsabilidade explícita. Assim, o projeto não se limita a utilizar IA como uma ferramenta auxiliar; procura demonstrar como a colaboração entre humano e IA pode ser estruturada de forma controlada e auditável.

No contexto da qualidade de software, a IA pode ser usada em várias atividades relevantes. Pode sugerir alterações de código, identificar potenciais defeitos, apoiar a geração de testes, explicar mensagens de erro, detetar padrões problemáticos e ajudar na documentação. Estas capacidades são particularmente úteis em processos de *Continuous Testing* e *Continuous Review*, onde as equipas procuram validar frequentemente o comportamento do sistema e rever alterações antes da sua integração. No entanto, a presença da IA nestes processos altera a forma como a responsabilidade deve ser entendida. Quando uma sugestão é produzida por uma ferramenta de IA e posteriormente integrada no sistema, a decisão final não pertence à ferramenta, mas sim ao elemento humano ou à equipa que validou essa sugestão.

Esta distinção é essencial para evitar uma visão simplista da responsabilidade. Uma falha de qualidade não deve ser atribuída apenas à IA nem apenas ao programador que aceitou a sugestão. Em processos reais, a responsabilidade tende a estar distribuída por vários níveis: a ferramenta que gerou a sugestão, o programador que a analisou, a equipa que reviu a alteração, a organização que definiu as regras de utilização da IA e, em certos contextos, normas ou orientações externas que regulam a adoção destas tecnologias. Esta foi uma das principais ideias do modelo conceptual apresentado no seminário.

O modelo conceptual do seminário organizava a responsabilidade em cinco camadas: ferramenta de IA, programador, equipa de revisão, organização e governação externa. A ferramenta de IA tem o papel de produzir sugestões, análises ou recomendações, mas não deve possuir autoridade final de decisão. O programador é responsável por interpretar e validar tecnicamente essas sugestões. A equipa de revisão deve funcionar como mecanismo adicional de controlo, discutindo riscos e confirmando decisões. A organização deve definir políticas, limites e critérios de utilização da IA. Por fim, a governação externa fornece princípios, normas e orientações que ajudam a enquadrar a utilização responsável destes sistemas.

O CollabQA foi desenhado precisamente para refletir estas camadas de responsabilidade num protótipo funcional. A camada correspondente à ferramenta de IA é representada pelo módulo de revisão assistida, que analisa o código e identifica riscos relacionados com segurança, fiabilidade e manutenibilidade. A camada do programador é representada pela validação humana obrigatória, onde cada sugestão deve ser aceite, rejeitada, aceite parcialmente ou enviada para revisão. A camada da equipa de revisão é simulada através do registo da decisão e da justificação associada, permitindo que outro elemento compreenda por que razão determinada sugestão foi aceite ou rejeitada. A camada organizacional é representada pelo processo definido no projeto, com gestão em Jira, tarefas estruturadas, evidências de teste e integração contínua. Por fim, a camada de governação externa é refletida na preocupação com rastreabilidade, auditoria e responsabilidade explícita.

No âmbito do enunciado do projeto, a colaboração Humano-IA foi também analisada através dos modelos *Human-in-the-Loop* (HITL) e *Human-on-the-Loop* (HOTL). No modelo HITL, o humano participa diretamente no processo de decisão, validando individualmente as sugestões antes de estas avançarem para etapas posteriores. No modelo HOTL, o humano atua mais como supervisor global do sistema, observando resultados, padrões de decisão, métricas e registos de auditoria, intervindo quando necessário. No CollabQA, o HITL é representado pela camada de decisão humana, onde cada sugestão exige aceitação, rejeição ou aceitação parcial. O HOTL é aproximado através da supervisão global proporcionada pelo audit log, pelas métricas finais e pela comparação experimental entre modos de trabalho. Esta distinção permite relacionar a implementação com as preocupações do enunciado sobre supervisão humana, confiança calibrada e riscos de supervisão reduzida.

Outro conceito central do seminário foi o de confiança calibrada. Confiar totalmente numa ferramenta de IA pode levar à aceitação acrítica de respostas plausíveis mas incorretas. Por outro lado, rejeitar completamente estas ferramentas impede que sejam aproveitadas para melhorar produtividade, cobertura de testes e capacidade de análise. A confiança calibrada corresponde a uma posição intermédia: a IA é usada como apoio, mas as suas sugestões continuam sujeitas a validação humana, testes e revisão. Esta lógica está presente em todo o funcionamento do CollabQA. A IA identifica riscos, mas não corrige automaticamente o sistema nem aprova decisões. A decisão final é sempre humana e deve ser acompanhada por uma justificação.

A necessidade de validação é particularmente importante porque os testes existentes nem sempre cobrem todos os cenários críticos. Um sistema pode apresentar uma cobertura aparentemente aceitável e, ainda assim, conter defeitos relevantes. No

caso de estudo deste projeto, os testes iniciais passavam com sucesso, mas não verificavam o comportamento do sistema perante sessões expiradas. Isto criava uma falsa sensação de qualidade. A análise assistida por IA permitiu identificar esse risco, mas a sugestão só ganhou valor depois de ser validada por um humano e transformada num teste concreto. Este processo demonstra que a IA pode apoiar a qualidade, mas que a confirmação do problema exige evidência executável, neste caso sob a forma de um teste automatizado.

A rastreabilidade foi outro princípio retirado do seminário. Em processos de desenvolvimento assistidos por IA, é importante saber quando a IA foi usada, que sugestão foi produzida, quem a avaliou, que decisão foi tomada e qual foi a justificação. Sem este registo, torna-se difícil reconstruir a cadeia de decisão em caso de falha. Por isso, o CollabQA inclui um *audit log*, onde ficam registadas as sugestões da IA, o risco associado, a decisão humana, a justificação e o responsável. Este registo permite transformar uma utilização informal da IA num processo documentado e auditável.

A opção por um módulo de autenticação como caso de estudo também resulta diretamente do seminário. No relatório anterior, foi discutido um cenário em que uma ferramenta de IA sugere uma alteração aparentemente simples num módulo de autenticação, os testes existentes continuam a passar, mas a alteração introduz uma falha relacionada com sessões expiradas. O projeto atual concretiza essa situação através de um módulo de autenticação simples, contendo login, validação de sessão e permissões. O defeito controlado encontra-se na função responsável por validar sessões, que inicialmente verificava apenas se o token existia, sem confirmar se a data de expiração já tinha sido ultrapassada.

Esta ligação ao caso de estudo é importante porque permite transformar uma discussão teórica numa experiência prática. O projeto não analisa a colaboração Humano-IA de forma abstrata; aplica-a a uma situação concreta, tecnicamente verificável e relacionada com um atributo crítico de qualidade: a segurança. Uma falha na validação de sessões pode permitir que um utilizador continue autenticado mesmo depois de a sua sessão já dever ter expirado. Assim, o caso de estudo demonstra como uma omissão aparentemente pequena pode ter impacto relevante na qualidade e segurança do sistema.

Além da ligação conceptual ao seminário, o CollabQA também responde aos objetivos definidos para o projeto da unidade curricular. O enunciado do projeto propõe o tema *Human-AI Collaboration in Software Quality* e destaca a importância de compreender e aplicar o ciclo de desenvolvimento com foco em *Continuous Testing* e *Continuous Review*. Também refere a utilização de ferramentas de IA para melhorar automação de testes, revisão de código e depuração. O CollabQA aborda diretamente estes pontos através de revisão assistida por IA, sugestão de testes, validação humana, execução automática de testes com *pytest*, medição de cobertura com *coverage.py* e integração contínua através de GitHub Actions.

A gestão do projeto também foi organizada de acordo com o enunciado, que recomenda a utilização de uma ferramenta como Jira, Trello ou Notion. Foi criado um quadro em Jira com tarefas relacionadas com definição de objetivos, arquitetura, implementação, testes, métricas, interface, integração contínua, análise de resultados

e relatório. Esta organização permitiu acompanhar a evolução do trabalho e associar as atividades práticas aos objetivos do projeto.

Em síntese, esta secção estabelece a ponte entre o seminário e o projeto. O seminário forneceu a base conceptual: a IA deve ser usada com confiança calibrada, validação humana, rastreabilidade e responsabilidade distribuída. O projeto CollabQA transforma essa base numa solução prática, aplicada a um módulo de autenticação com defeito controlado. Desta forma, o trabalho mostra que a colaboração Humano-IA pode contribuir para a qualidade de software, desde que seja integrada num processo onde as sugestões da IA são tratadas como hipóteses, as decisões são humanas, os testes fornecem evidência e a responsabilidade fica registada.

3 Caso de Estudo e Formulação do Problema

O caso de estudo utilizado neste projeto foi construído a partir do cenário discutido no seminário: uma equipa de desenvolvimento utiliza IA para apoiar alterações num módulo de autenticação, os testes existentes continuam a passar, mas permanece uma falha crítica relacionada com a validação de sessões expiradas. A escolha deste caso de estudo não foi aleatória. A autenticação é uma parte sensível de qualquer sistema de software, porque está diretamente relacionada com segurança, controlo de acesso, gestão de sessões e proteção de utilizadores. Assim, uma pequena falha nesta área pode ter consequências significativas para a qualidade global do sistema.

O objetivo do caso de estudo foi criar um ambiente pequeno, controlado e tecnicamente verificável, onde fosse possível observar a colaboração entre humano e IA em tarefas de qualidade de software. Em vez de utilizar uma aplicação grande e difícil de controlar, foi desenvolvido um módulo de autenticação simples em Python. Este módulo contém funcionalidades essenciais, como autenticação de utilizadores, criação de sessões, validação de tokens, encerramento de sessão e verificação de permissões. Esta simplicidade permitiu isolar o problema principal e demonstrar, de forma clara, como uma falha pode passar despercebida mesmo quando os testes existentes passam com sucesso.

A Tabela 1 apresenta as principais funções do módulo de autenticação usado no caso de estudo.

Table 1 Principais funções do módulo de autenticação

Função	Descrição
<code>login(username, password)</code>	Verifica as credenciais do utilizador e cria um token de sessão quando a autenticação é bem-sucedida.
<code>logout(token)</code>	Remove uma sessão ativa associada a um determinado token.
<code>validate_session(token)</code>	Verifica se uma sessão é válida. Na versão inicial, continha o defeito controlado do caso de estudo.
<code>has_permission(username, permission)</code>	Verifica se um utilizador possui uma determinada permissão.
<code>reset_sessions()</code>	Limpa as sessões ativas, sendo usada principalmente para garantir independência entre testes.

O defeito principal foi introduzido na função `validate_session`. Na implementação inicial, esta função apenas verificava se o token existia na estrutura de sessões ativas. No entanto, não verificava se a data de expiração da sessão já tinha sido ultrapassada. Como consequência, uma sessão expirada poderia continuar a ser considerada válida, desde que o token ainda existisse no dicionário de sessões. Este comportamento representa uma falha de segurança e de correção funcional, porque permite que uma sessão continue ativa para além do período previsto.

Este defeito é particularmente relevante porque não causa necessariamente uma falha imediata nos testes básicos. Um teste que apenas verifica se uma sessão recém-criada é válida continua a passar. Da mesma forma, um teste que verifica se o logout remove uma sessão também continua a passar. O problema só aparece quando se testa explicitamente o cenário em que a sessão existe, mas a sua data de expiração já está no passado. Isto mostra a diferença entre ter testes que passam e ter testes que cobrem os riscos realmente importantes.

Inicialmente, foram criados testes unitários simples para validar o comportamento básico do módulo. Estes testes tinham como objetivo representar uma suíte inicial incompleta, semelhante à situação descrita no seminário: os testes existentes davam uma aparência de qualidade, mas não cobriam um caso crítico. A Tabela 2 apresenta os testes iniciais.

Table 2 Testes iniciais do caso de estudo

Teste	Objetivo
<code>test_login_success</code>	Verificar se o login com credenciais válidas cria um token de sessão.
<code>test_login_fail_with_wrong_password</code>	Verificar se o login falha quando a palavra-passe está incorreta.
<code>test_logout_removes_session</code>	Verificar se o logout remove corretamente uma sessão ativa.
<code>test_regular_user_has_read_permission</code>	Verificar se um utilizador comum tem permissão de leitura.
<code>test_regular_user_does_not_have_delete_permission</code>	Verificar se um utilizador comum não tem permissão de eliminação.

A execução inicial destes testes resultou em cinco testes bem-sucedidos. Este resultado poderia levar a equipa a concluir que o módulo estava correto. No entanto, essa conclusão seria precipitada, porque nenhum dos testes verificava o comportamento de uma sessão expirada. Esta situação foi intencionalmente criada para demonstrar uma limitação comum em processos de teste: uma suíte de testes pode passar totalmente e, ainda assim, não cobrir cenários importantes. Assim, o caso de estudo permite mostrar que a qualidade de software não deve ser avaliada apenas pelo facto de os testes existentes passarem, mas também pela relevância dos cenários testados.

A formulação do problema pode ser resumida através da diferença entre o comportamento esperado e o comportamento observado na implementação inicial. Esta diferença é apresentada na Tabela 3.

Table 3 Formulação do defeito relacionado com sessões expiradas

Elemento	Descrição
Contexto	O utilizador inicia sessão com credenciais válidas e recebe um token associado a uma data de expiração.
Condição crítica	O token continua guardado na estrutura de sessões, mas a data de expiração já foi ultrapassada.
Comportamento esperado	A função <code>validate_session</code> deve devolver <code>False</code> , porque a sessão já expirou.
Comportamento observado	A função devolvia <code>True</code> , porque apenas verificava se o token existia.
Impacto	Uma sessão expirada podia continuar a ser aceite, criando um risco de segurança e de controlo de acesso.
Correção necessária	A função de validação deve verificar simultaneamente se o token existe e se a data de expiração ainda está no futuro.

A questão principal do projeto surge precisamente deste problema. Se os testes iniciais passam e o defeito não é imediatamente visível, de que forma pode a colaboração entre humano e IA contribuir para melhorar o processo de qualidade? O CollabQA foi desenvolvido para responder a esta questão. O protótipo simula uma camada de análise assistida por IA que identifica riscos no código, incluindo o facto de sessões expiradas poderem continuar a ser aceites. No entanto, a sugestão da IA não é tratada como decisão final. Um humano deve validar a sugestão, justificar a decisão e registar essa avaliação.

Depois da validação humana, o sistema sugere um novo teste específico para o risco identificado. Esse teste verifica explicitamente se uma sessão expirada é rejeitada. Quando o teste foi adicionado à versão inicial do módulo, a execução resultou em uma falha: cinco testes passaram e um teste falhou. Este resultado confirmou que o defeito existia. Em seguida, a função `validate_session` foi corrigida para verificar a data de expiração. Após a correção, os seis testes passaram com sucesso.

Desta forma, o caso de estudo permite observar o ciclo completo proposto pelo projeto: a IA identifica um risco, o humano valida a sugestão, o sistema propõe um teste, o teste confirma o defeito, o código é corrigido e a nova execução dos testes fornece evidência de melhoria. Este ciclo representa a essência da colaboração Humano-IA defendida neste trabalho. A IA aumenta a capacidade de análise, mas a decisão permanece humana e a confirmação do problema é feita através de testes automatizados.

Além do defeito principal, a análise assistida por IA identificou outros dois riscos: a atribuição automática de permissões ao utilizador administrador e a geração previsível de tokens de sessão. Estes riscos foram incluídos para enriquecer o caso de estudo e demonstrar que a IA pode contribuir para uma análise mais ampla do código. No entanto, o foco experimental principal permaneceu na falha da sessão expirada, por ser o problema mais diretamente relacionado com o cenário do seminário e por permitir uma validação objetiva através de um teste automatizado.

A formulação do problema pode, assim, ser sintetizada na seguinte questão de investigação prática: a colaboração entre humano e IA melhora a deteção de defeitos, a

criação de testes úteis e a rastreabilidade das decisões num módulo de autenticação? As secções seguintes apresentam a arquitetura e a implementação do CollabQA, bem como a metodologia experimental usada para comparar três modos de trabalho: humano sozinho, IA sozinha e colaboração Humano-IA.

4 Arquitetura da Solução

A arquitetura do CollabQA foi definida com o objetivo de representar, de forma prática, a colaboração entre humano e inteligência artificial (IA) em atividades de qualidade de software. A solução foi organizada segundo uma arquitetura em camadas, uma vez que este estilo permite separar responsabilidades, reduzir dependências entre componentes e tornar mais clara a fronteira entre análise automatizada, decisão humana, execução de testes e registo de evidência.

A escolha de uma arquitetura em camadas é adequada para este projeto por três razões principais. Em primeiro lugar, permite isolar a camada de IA da restante aplicação. Isto é importante porque a ferramenta de análise pode ser substituída no futuro por um modelo de linguagem real sem obrigar a reescrever os módulos de decisão humana, auditoria ou métricas. Em segundo lugar, permite distinguir claramente entre sugestões geradas automaticamente e decisões tomadas por humanos. Esta separação é essencial para o tema do projeto, uma vez que a IA não deve possuir autoridade final sobre alterações de código ou validações de qualidade. Em terceiro lugar, facilita a rastreabilidade, porque cada etapa do processo produz artefactos próprios, como ficheiros JSON, registos CSV, resultados de testes e relatórios de métricas.

A Figura 1 apresenta uma visão geral da arquitetura da solução. O fluxo começa com o módulo de autenticação usado como caso de estudo. Em seguida, a camada de assistência por IA analisa o código e gera sugestões de revisão. Essas sugestões são enviadas para a camada de decisão humana, onde o revisor aceita, rejeita ou aceita parcialmente cada recomendação. As decisões validadas alimentam a camada de sugestão de testes, que propõe novos testes associados aos riscos identificados. Por fim, a camada de métricas e auditoria consolida os resultados, incluindo testes executados, cobertura, decisões humanas e entradas no audit log.

A primeira camada corresponde ao módulo de autenticação. Este módulo representa o sistema sob análise e contém as funções necessárias para simular um cenário de autenticação simples. A função mais importante para o caso de estudo é `validate_session`, uma vez que a versão inicial continha o defeito controlado relacionado com sessões expiradas. Esta camada não representa a ferramenta CollabQA em si, mas sim o objeto de análise sobre o qual as restantes camadas atuam.

A segunda camada é a camada de assistência por IA. No protótipo desenvolvido, esta camada foi implementada através do módulo `ai_reviewer.py`. Este módulo analisa o código-fonte e produz sugestões semelhantes às que poderiam ser geradas por uma ferramenta de IA usada em contexto de revisão de código. A análise identifica riscos relacionados com segurança, fiabilidade e manutenibilidade. No caso de estudo, foram produzidas três sugestões principais: a possibilidade de sessões expiradas continuarem a ser aceites, a atribuição automática de permissões ao administrador e a previsibilidade dos tokens de sessão.

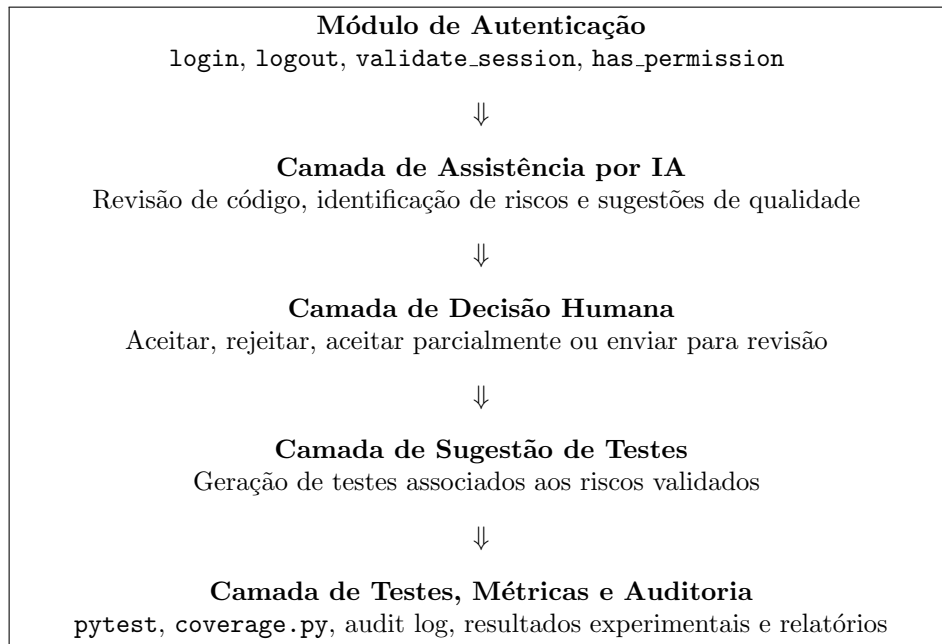


Fig. 1 Arquitetura em camadas do CollabQA.

Importa destacar que, nesta versão do protótipo, a componente de IA foi implementada de forma determinística e controlada. Esta decisão foi tomada para garantir reprodutibilidade experimental. Como o objetivo do projeto é estudar o processo de colaboração Humano-IA, e não comparar modelos de linguagem, optou-se por simular a camada de IA com regras explícitas. Esta abordagem permite repetir a experiência várias vezes e obter sempre as mesmas sugestões. Numa versão futura, esta camada poderia ser substituída por uma chamada a um modelo de linguagem real, mantendo a restante arquitetura praticamente inalterada.

A terceira camada é a camada de decisão humana. Esta é uma das partes mais importantes do projeto, porque representa a passagem de uma sugestão automática para uma decisão validada. O módulo `human_decision.py` obriga o revisor a analisar cada sugestão e a escolher uma decisão: aceitar, rejeitar, aceitar parcialmente ou enviar para revisão. Além disso, cada decisão exige uma justificação textual e a identificação do responsável. Esta camada concretiza o princípio de que a IA pode apoiar a análise, mas a decisão final deve permanecer humana.

A quarta camada é a camada de sugestão de testes. Esta camada foi implementada no módulo `test_generator.py` e utiliza as decisões humanas como entrada. Ou seja, os testes são sugeridos com base nos riscos que foram aceites ou parcialmente aceites pelo humano. Esta opção é relevante porque impede que qualquer sugestão da IA seja automaticamente transformada em teste sem validação. No caso principal, a sugestão AI-SEC-001, relacionada com sessões expiradas, originou o teste TEST-SEC-001, cujo objetivo era verificar se uma sessão expirada era rejeitada.

A quinta camada corresponde aos testes, métricas e auditoria. Esta camada reúne vários artefactos produzidos ao longo do processo. Os testes automatizados foram executados com `pytest`, enquanto a cobertura foi medida com `coverage.py`. As decisões humanas foram guardadas em ficheiros JSON e também num ficheiro CSV de auditoria. As métricas finais foram calculadas pelo módulo `metrics.py`, enquanto a comparação entre os modos humano sozinho, IA sozinha e Humano-IA foi produzida pelo módulo `experiment_summary.py`. Esta camada permite transformar o processo de colaboração numa avaliação observável e documentada.

A Tabela 4 resume as camadas da arquitetura, os módulos correspondentes e as respetivas responsabilidades.

Table 4 Camadas da arquitetura do CollabQA

Camada	Módulos/artefactos	Responsabilidade principal
Sistema sob análise	<code>sample_project/auth.py</code>	Representar o módulo de autenticação com funcionalidades de login, sessões e permissões.
Assistência por IA	<code>ai_reviewer.py</code>	Analisar o código e gerar sugestões de revisão associadas a riscos de qualidade.
Decisão humana	<code>human_decision.py</code>	Validar sugestões da IA através de decisões justificadas e associadas a um responsável.
Sugestão de testes	<code>test_generator.py</code>	Propor testes automatizados com base nos riscos validados pelo humano.
Auditoria	<code>audit_logger.py</code> , <code>audit_log.csv</code>	Registar sugestões, decisões, justificações, responsáveis e níveis de risco.
Métricas e resultados	<code>metrics.py</code> , <code>experiment_summary.py</code>	Calcular métricas, consolidar resultados e comparar modos de trabalho.
Interface	<code>main.py</code>	Disponibilizar uma interface Streamlit para demonstração do fluxo completo.
Integração contínua	GitHub Actions	Executar testes automaticamente após alterações no repositório.

A arquitetura também foi pensada para apoiar o ciclo de *Continuous Testing*. A execução dos testes com `pytest` permite verificar automaticamente se o comportamento esperado do módulo é preservado. A medição com `coverage.py` fornece uma indicação quantitativa sobre a cobertura dos testes. A integração com GitHub Actions permite executar estes testes sempre que existem alterações no repositório, aproximando o projeto de um fluxo real de integração contínua. Embora o sistema seja pequeno, esta configuração demonstra o princípio de validação automática frequente.

Do ponto de vista de *Continuous Review*, o CollabQA permite observar como uma alteração ou risco identificado pela IA pode ser submetido a revisão humana antes de ser considerado válido. O processo não termina na sugestão automática. Pelo contrário, a arquitetura força uma etapa de validação humana e registo da justificação. Esta decisão é coerente com a motivação do projeto, uma vez que a colaboração Humano-IA só contribui para a qualidade se existir controlo sobre as recomendações produzidas.

A camada de auditoria desempenha um papel central na ligação entre arquitetura e responsabilidade. Cada decisão humana fica registada com o identificador da sugestão, categoria, nível de risco, título, decisão, justificação, responsável e indicação de que a validação humana era necessária. Este registo permite reconstruir a cadeia de decisão e perceber de que forma determinada sugestão da IA foi tratada. Assim, o audit log não é apenas um ficheiro auxiliar; é um mecanismo de accountability.

A interface Streamlit foi incluída para facilitar a demonstração e a interpretação do fluxo. A interface está organizada em cinco separadores: revisão de código, decisões humanas, sugestões de testes, métricas e audit log. Esta organização reflete diretamente a arquitetura da solução. O primeiro separador mostra as sugestões produzidas pela IA. O segundo permite validar essas sugestões. O terceiro apresenta os testes sugeridos. O quarto mostra as métricas de avaliação. O quinto apresenta as decisões registadas no audit log. Desta forma, a interface torna visível a cadeia completa de colaboração.

Em termos de artefactos produzidos, a arquitetura gera vários ficheiros relevantes para a avaliação. As sugestões da IA são guardadas em `data/ai_suggestions.json`. As decisões humanas são guardadas em `data/decisions.json`. O audit log é guardado em `data/audit_log.csv`. As sugestões de testes são guardadas em `data/test_suggestions.json`. Os resultados de testes e cobertura são guardados na pasta `reports/`. Finalmente, a comparação experimental entre modos de trabalho é guardada em `reports/experiment_comparison.csv` e `reports/experiment_comparison.json`. Esta separação de artefactos facilita a análise posterior e reforça a rastreabilidade.

A arquitetura adotada apresenta também algumas limitações assumidas. Por se tratar de um protótipo académico, a camada de IA não utiliza um modelo real em tempo de execução, mas uma análise determinística. Esta decisão aumenta a reprodutibilidade, mas reduz a variabilidade típica de sistemas baseados em modelos de linguagem. Além disso, o sistema foi aplicado apenas a um módulo pequeno de autenticação. Ainda assim, a arquitetura foi desenhada de forma modular, permitindo extensão futura para outros tipos de código, outras linguagens, integração com modelos de IA reais e análise de projetos maiores.

Em síntese, a arquitetura do CollabQA foi desenhada para transformar os princípios discutidos no seminário numa solução prática e observável. A separação entre assistência por IA, decisão humana, sugestão de testes, auditoria e métricas permite demonstrar que a IA pode apoiar tarefas de qualidade de software sem eliminar a necessidade de validação humana. A solução torna explícita a cadeia de decisão e fornece evidência através de testes, cobertura, logs e resultados experimentais.

5 Implementação

A implementação do CollabQA foi realizada em Python, com o objetivo de criar um protótipo simples, reprodutível e suficientemente modular para demonstrar o processo de colaboração Humano-IA em atividades de qualidade de software. A solução foi estruturada em torno de um caso de estudo de autenticação, de uma camada de

análise assistida por IA, de uma camada de decisão humana, de mecanismos de auditoria, de geração de sugestões de testes, de cálculo de métricas e de uma interface de demonstração em Streamlit.

A implementação não teve como objetivo criar uma aplicação de autenticação completa para uso real em produção. O objetivo foi diferente: construir um ambiente controlado onde fosse possível observar um problema de qualidade, aplicar análise assistida por IA, validar essa análise por intervenção humana, gerar testes associados ao risco identificado e medir o impacto da correção. Esta decisão permitiu manter o projeto focado nos objetivos da unidade curricular, nomeadamente qualidade de software, revisão contínua, testes contínuos, rastreabilidade e colaboração entre humano e IA.

A Figura 2 apresenta o fluxo operacional implementado no CollabQA. Este fluxo representa a sequência real seguida durante o projeto: primeiro é analisado o módulo de autenticação; depois são geradas sugestões de revisão; em seguida, o humano valida essas sugestões; posteriormente, são geradas sugestões de testes; finalmente, os testes, métricas e logs são usados para avaliar o impacto do processo.

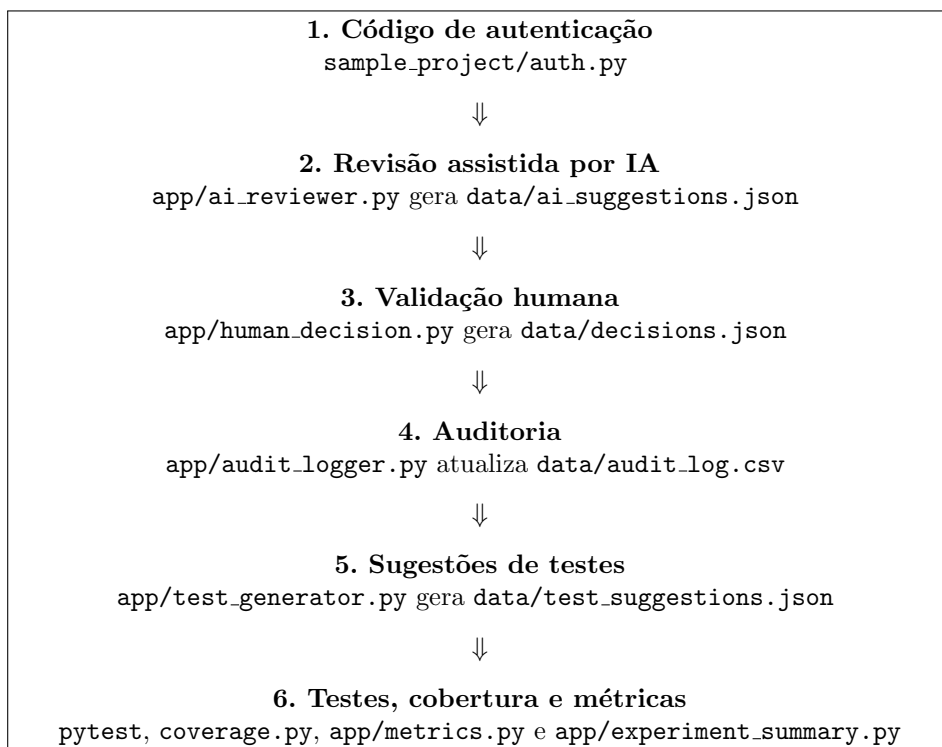


Fig. 2 Fluxo operacional implementado no CollabQA.

5.1 Estrutura do Projeto

O projeto foi organizado em várias pastas, cada uma com uma responsabilidade distinta. A pasta `sample_project/` contém o módulo de autenticação usado como caso de estudo. A pasta `app/` contém os módulos principais do CollabQA. A pasta `data/` guarda os artefactos gerados durante a análise, como sugestões da IA, decisões humanas, sugestões de testes e audit log. A pasta `reports/` contém outputs de testes, cobertura, métricas e comparação experimental. A pasta `experiments/` contém os dados usados para comparar os três modos de avaliação: humano sozinho, IA sozinha e Humano-IA.

A Tabela 5 apresenta os ficheiros principais implementados e a sua função no projeto.

Table 5 Ficheiros principais implementados no projeto

Ficheiro	Função
<code>sample_project/auth.py</code>	Implementa o módulo de autenticação usado como caso de estudo, incluindo login, logout, validação de sessão e permissões.
<code>sample_project/tests/test_auth.py</code>	Contém os testes automatizados do módulo de autenticação, incluindo o teste que valida sessões expiradas.
<code>app/ai_reviewer.py</code>	Analisa o código e gera sugestões de revisão semelhantes às que poderiam ser produzidas por uma ferramenta de IA.
<code>app/human_decision.py</code>	Permite ao humano validar cada sugestão da IA, escolhendo uma decisão e escrevendo uma justificação.
<code>app/audit_logger.py</code>	Regista as decisões humanas num ficheiro CSV de auditoria.
<code>app/test_generator.py</code>	Gera sugestões de testes com base nas sugestões da IA que foram aceites ou parcialmente aceites pelo humano.
<code>app/metrics.py</code>	Calcula métricas finais, incluindo sugestões, decisões, testes, audit log e cobertura.
<code>app/experiment_summary.py</code>	Compara os modos humano sozinho, IA sozinha e Humano-IA.
<code>app/main.py</code>	Implementa a interface Streamlit usada para demonstrar o fluxo completo do CollabQA.
<code>pytest.ini</code>	Define a pasta onde os testes automatizados devem ser procurados pelo <code>pytest</code> .
<code>.github/workflows/tests.yml</code>	Configura o GitHub Actions para executar testes e cobertura automaticamente.

5.2 Módulo de Autenticação

O módulo de autenticação foi implementado no ficheiro `sample_project/auth.py`. Este módulo contém uma estrutura simples de utilizadores, com palavras-passe e permissões, e uma estrutura em memória para guardar sessões ativas. A função `login` verifica as credenciais e cria um token de sessão. A função `logout` remove uma sessão ativa. A função `has_permission` verifica permissões associadas a um utilizador. A

função `reset_sessions` foi criada para garantir que os testes são independentes entre si.

A função mais importante para o caso de estudo é `validate_session`. Na versão inicial, esta função continha o defeito controlado do projeto: apenas verificava se o token existia na estrutura de sessões, sem verificar se a data de expiração já tinha sido ultrapassada. A versão inicial tinha a seguinte lógica conceptual:

```
def validate_session(token):  
    return token in SESSIONS
```

Esta implementação era insuficiente, porque uma sessão expirada continuava a ser aceite enquanto o token permanecesse guardado. Depois da identificação do problema e da validação humana da sugestão da IA, a função foi corrigida para verificar também a data de expiração:

```
def validate_session(token):  
    session = SESSIONS.get(token)  
  
    if session is None:  
        return False  
  
    return datetime.now() < session["expires_at"]
```

Esta correção altera o comportamento da função de forma significativa. Uma sessão só é considerada válida se o token existir e se a data atual ainda for anterior à data de expiração. Assim, o comportamento passa a estar alinhado com o requisito esperado para gestão de sessões.

5.3 Testes Automatizados

Os testes automatizados foram implementados com `pytest`. Inicialmente, foram criados cinco testes para validar comportamentos básicos do módulo de autenticação: login bem-sucedido, login com palavra-passe incorreta, remoção de sessão por logout, permissão de leitura para utilizador comum e ausência de permissão de eliminação para utilizador comum.

Estes testes iniciais passavam com sucesso, mas não verificavam o cenário crítico de sessão expirada. Este ponto foi essencial para o projeto, porque demonstrou que uma suíte de testes pode passar totalmente sem cobrir um risco relevante. Após a análise assistida por IA e a validação humana, foi adicionado o teste `test_expired_session_is_invalid`, que força a data de expiração de uma sessão para o passado e verifica se a função `validate_session` devolve `False`.

O teste adicionado foi o seguinte:

```
def test_expired_session_is_invalid():  
    token = login("admin", "admin123")  
    SESSIONS[token]["expires_at"] = datetime.now() - timedelta(minutes=1)
```

```
assert validate_session(token) is False
```

Quando este teste foi executado contra a versão inicial do módulo, falhou. O resultado esperado era `False`, mas a função devolvia `True`. Esta falha confirmou que o defeito existia. Após a correção da função `validate_session`, todos os seis testes passaram.

5.4 Revisão Assistida por IA

O módulo `app/ai_reviewer.py` implementa a camada de revisão assistida por IA. Nesta versão do protótipo, a análise é determinística e baseada em padrões do código. Esta opção foi escolhida para garantir reprodutibilidade. Em vez de depender de respostas variáveis de um modelo externo, o módulo produz sempre as mesmas sugestões para o mesmo código. Isto permite repetir a experiência e comparar os resultados de forma consistente.

O módulo analisa o ficheiro `sample_project/auth.py` e gera uma lista de sugestões em formato JSON. Cada sugestão contém um identificador, categoria, nível de risco, título, descrição, recomendação, atributos de qualidade afetados e indicação de que a validação humana é obrigatória. Foram geradas três sugestões principais:

- **AI-SEC-001:** sessões expiradas podem continuar a ser aceites;
- **AI-AUTH-002:** o utilizador administrador recebe permissões automaticamente;
- **AI-REL-003:** o token de sessão é previsível.

A sugestão mais relevante para o caso de estudo foi **AI-SEC-001**, porque identificou diretamente o problema da função `validate_session`. Esta sugestão foi classificada como risco alto e associada aos atributos de qualidade segurança, fiabilidade e correção.

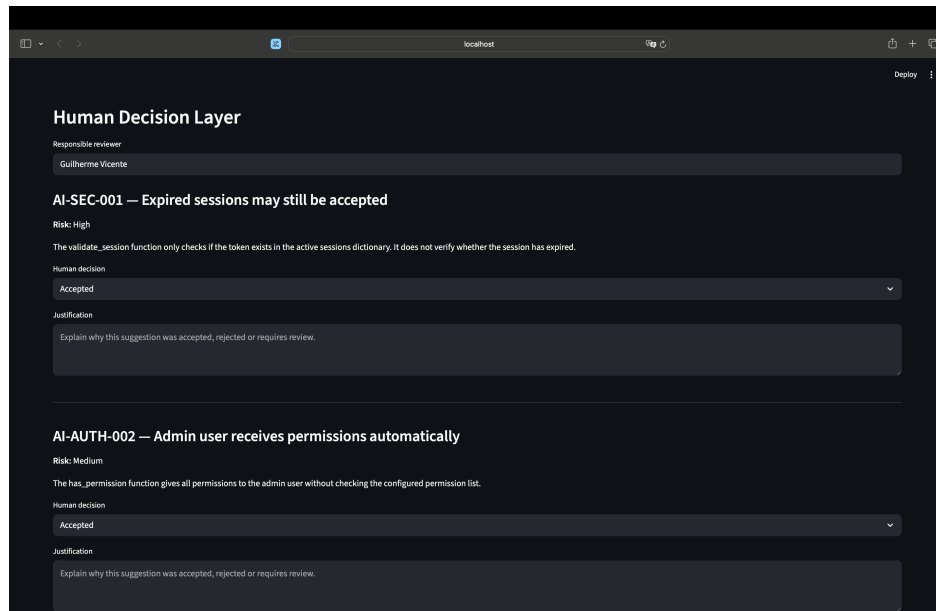
5.5 Decisão Humana e Audit Log

O módulo `app/human_decision.py` implementa a camada de decisão humana. Depois de as sugestões da IA serem geradas, o revisor deve analisar cada uma delas e escolher uma decisão. As opções disponíveis são: aceite, rejeitada, aceite parcialmente ou necessita de revisão humana. Além disso, cada decisão exige uma justificação textual e a identificação do responsável.

Esta implementação é central para o objetivo do projeto, porque impede que as sugestões da IA sejam tratadas como decisões automáticas. A IA propõe, mas o humano valida. No caso executado, as decisões foram as seguintes:

- **AI-SEC-001:** aceite, porque a função `validate_session` não verificava a expiração da sessão;
- **AI-AUTH-002:** aceite parcialmente, porque o comportamento do administrador podia ser intencional, mas deveria ser documentado e revisto;
- **AI-REL-003:** aceite, porque tokens previsíveis reduzem a segurança e podem causar problemas em múltiplas sessões.

O módulo `app/audit_logger.py` regista cada decisão num ficheiro CSV. Este ficheiro funciona como audit log e contém a data e hora da decisão, identificador da sugestão, categoria, risco, título, decisão humana, justificação, responsável e indicação de validação humana obrigatória. Este mecanismo transforma a decisão humana em evidência rastreável.



The screenshot shows a web interface titled "Human Decision Layer" running on a Streamlit application. At the top, there's a "Deploy" button. Below the title, a "Responsible reviewer" field is populated with "Guilherme Vicente". The interface displays two security suggestions for review:

- AI-SEC-001 — Expired sessions may still be accepted**
 - Risk:** High
 - Description:** The `validate_session` function only checks if the token exists in the active sessions dictionary. It does not verify whether the session has expired.
 - Human decision:** A dropdown menu showing "Accepted".
 - Justification:** A text area with the placeholder "Explain why this suggestion was accepted, rejected or requires review."
- AI-AUTH-002 — Admin user receives permissions automatically**
 - Risk:** Medium
 - Description:** The `has_permission` function gives all permissions to the admin user without checking the configured permission list.
 - Human decision:** A dropdown menu showing "Accepted".
 - Justification:** A text area with the placeholder "Explain why this suggestion was accepted, rejected or requires review."

Fig. 3 Interface Streamlit usada para validação humana das sugestões geradas pela IA. Cada sugestão exige uma decisão explícita e uma justificação por parte do revisor.

5.6 Geração de Sugestões de Testes

O módulo `app/test_generator.py` utiliza as decisões humanas como entrada. Isto significa que apenas sugestões aceites ou parcialmente aceites pelo humano são usadas para gerar propostas de testes. Esta decisão de desenho é importante porque mantém o humano no centro do processo. O sistema não transforma automaticamente qualquer sugestão da IA num teste; primeiro exige validação.

Para a sugestão AI-SEC-001, foi gerado o teste TEST-SEC-001, cujo objetivo era verificar se uma sessão expirada era rejeitada. Para a sugestão AI-AUTH-002, foi gerado um teste para documentar o comportamento do administrador. Para a sugestão AI-REL-003, foi gerado um teste relacionado com a previsibilidade dos tokens.

A Tabela 6 apresenta os principais artefactos gerados durante a execução do projeto.

Table 6 Artefactos gerados pelo CollabQA

Artefacto	Descrição
<code>data/ai_suggestions.json</code>	Guarda as sugestões geradas pela camada de assistência por IA.
<code>data/decisions.json</code>	Guarda as decisões humanas tomadas sobre cada sugestão.
<code>data/audit_log.csv</code>	Regista decisões, justificações, responsáveis, riscos e categorias.
<code>data/test_suggestions.json</code>	Guarda os testes sugeridos com base nos riscos validados.
<code>reports/failing_test_output.txt</code>	Guarda a saída da execução em que o teste da sessão expirada falhou.
<code>reports/passing_test_output.txt</code>	Guarda a saída da execução após correção, com todos os testes a passar.
<code>reports/coverage_after_fix.txt</code>	Guarda a cobertura final obtida depois da correção.
<code>reports/final_results.csv</code>	Guarda as métricas finais em formato tabular.
<code>reports/experiment_comparison.csv</code>	Guarda a comparação entre humano sozinho, IA sozinha e Humano-IA.

5.7 Métricas e Comparação Experimental

O módulo `app/metrics.py` calcula métricas a partir dos ficheiros gerados durante a execução do projeto. Entre as métricas calculadas encontram-se o número de sugestões da IA, o número de sugestões validadas por humanos, sugestões aceites, sugestões parcialmente aceites, sugestões rejeitadas, sugestões de alto risco, sugestões de testes, entradas no audit log, testes que falharam antes da correção, testes que passaram depois da correção e cobertura final.

Além disso, foi criado o módulo `app/experiment_summary.py`, responsável por comparar três modos de trabalho: humano sozinho, IA sozinha e Humano-IA. Esta comparação é baseada em métricas como problemas relevantes encontrados, falsos positivos, testes úteis sugeridos, decisões humanas validadas e entradas no audit log. O modo Humano-IA obteve a melhor pontuação, porque combinou identificação de problemas, sugestão de testes, validação humana e rastreabilidade.

5.8 Interface Streamlit

A interface de demonstração foi implementada com Streamlit no ficheiro `app/main.py`. A interface foi organizada em cinco separadores, correspondentes ao fluxo principal do CollabQA:

- revisão de código;
- decisões humanas;
- sugestões de testes;
- métricas;
- audit log.

No separador de revisão de código, o utilizador pode visualizar o módulo de autenticação e executar a análise assistida por IA. No separador de decisões humanas,

o utilizador pode validar cada sugestão, escolher uma decisão e escrever uma justificação. No separador de sugestões de testes, são apresentadas as propostas de testes associadas aos riscos validados. No separador de métricas, são apresentados os resultados calculados pelo sistema. No separador de audit log, é apresentada a tabela de decisões registadas.

A interface não foi concebida como uma aplicação final de produção, mas como uma ferramenta de demonstração e validação conceptual. O seu objetivo é tornar visível a cadeia de colaboração Humano-IA, permitindo mostrar facilmente, durante a apresentação, como a sugestão da IA se transforma numa decisão humana, num teste e numa evidência de correção.

5.9 Integração Contínua

Para demonstrar o princípio de *Continuous Testing*, foi configurado um workflow de GitHub Actions. Este workflow executa automaticamente os testes com `pytest` e calcula a cobertura com `coverage.py` sempre que existe um *push* ou um *pull request*. A configuração encontra-se no ficheiro `.github/workflows/tests.yml`.

A Tabela 7 resume os principais comandos usados para executar o projeto e gerar os artefactos.

Table 7 Comandos principais usados no projeto

Comando	Função
<code>pytest</code>	Executa os testes automatizados do módulo de autenticação.
<code>coverage run -m pytest</code>	Executa os testes recolhendo dados de cobertura.
<code>coverage report</code>	Apresenta a cobertura final no terminal.
<code>python app/ai_reviewer.py</code>	Gera sugestões de revisão assistida por IA.
<code>sample_project/auth.py</code>	
<code>python app/human_decision.py</code>	Permite validar humanamente as sugestões da IA.
<code>python app/test_generator.py</code>	Gera sugestões de testes com base nas decisões humanas.
<code>sample_project/auth.py</code>	
<code>python app/metrics.py</code>	Calcula as métricas finais do projeto.
<code>python app/experiment_summary.py</code>	Gera a comparação experimental entre modos de trabalho.
<code>streamlit run app/main.py</code>	Inicia a interface visual do CollabQA.

A implementação resultante permite demonstrar o ciclo completo do projeto: o código é analisado, os riscos são identificados, as sugestões são validadas por um humano, as decisões são registadas, os testes são sugeridos, o defeito é confirmado, a correção é implementada e os resultados são avaliados. Este ciclo traduz, de forma prática, a ideia central do trabalho: a IA pode apoiar a qualidade de software, mas o seu valor depende da validação humana, da existência de testes e da rastreabilidade das decisões.

6 Metodologia Experimental

A metodologia experimental foi definida para avaliar se a colaboração entre humano e inteligência artificial (IA) contribui para melhorar atividades de qualidade de software num módulo de autenticação. O objetivo não foi realizar um estudo industrial de larga escala, mas sim uma avaliação controlada, reprodutível e alinhada com o caso de estudo desenvolvido no projeto. Desta forma, a experiência permite observar, num ambiente pequeno mas tecnicamente verificável, como diferentes modos de trabalho afetam a detecção de defeitos, a sugestão de testes e a rastreabilidade das decisões.

A questão experimental principal foi formulada da seguinte forma: a colaboração entre humano e IA melhora a detecção de defeitos, a criação de testes úteis e a rastreabilidade das decisões num módulo de autenticação? Esta questão está diretamente relacionada com o objetivo do projeto, uma vez que o CollabQA pretende demonstrar que a IA pode apoiar atividades de qualidade de software, mas que o seu valor depende da validação humana e da existência de evidência verificável.

A avaliação foi organizada em torno de três modos de trabalho: humano sozinho, IA sozinha e colaboração Humano-IA. Esta comparação foi escolhida porque permite distinguir o contributo de cada elemento do processo. O modo humano sozinho representa uma revisão manual tradicional. O modo IA sozinha representa a utilização de análise automatizada sem validação humana. O modo Humano-IA representa o fluxo completo proposto pelo CollabQA, no qual a IA sugere, o humano valida, os testes são gerados com base nas decisões e os resultados são registados.

Apesar de a comparação operacional ter sido organizada em três modos de trabalho, estes modos foram também interpretados à luz dos modelos HITL e HOTL. O modo Humano-IA corresponde à condição mais próxima de HITL, uma vez que cada sugestão gerada pela camada de revisão assistida exige uma decisão humana explícita antes de avançar para a geração de testes. O modo IA sozinha representa uma condição de supervisão reduzida, usada para observar o que se perde quando as sugestões automáticas não são avaliadas individualmente por um humano. Esta condição aproxima-se da lógica HOTL, não como implementação industrial completa, mas como uma forma controlada de supervisão global baseada em métricas e audit log. O modo humano sozinho foi mantido como referência de comparação, permitindo observar o contributo da revisão manual sem assistência automática.

A Figura 4 apresenta o fluxo geral da metodologia experimental.

6.1 Objetivos da Avaliação

A avaliação experimental teve quatro objetivos principais. O primeiro objetivo foi verificar se o defeito relacionado com sessões expiradas podia ser identificado a partir da análise do código. O segundo objetivo foi observar se a IA conseguia sugerir riscos úteis para a revisão de qualidade. O terceiro objetivo foi analisar se a validação humana acrescentava valor ao processo, distinguindo sugestões aceites, parcialmente aceites ou rejeitadas. O quarto objetivo foi verificar se o processo completo gerava evidência rastreável através de testes, métricas e audit log.

Estes objetivos refletem a preocupação central do projeto: a qualidade de software não depende apenas da existência de testes, mas também da capacidade de identificar

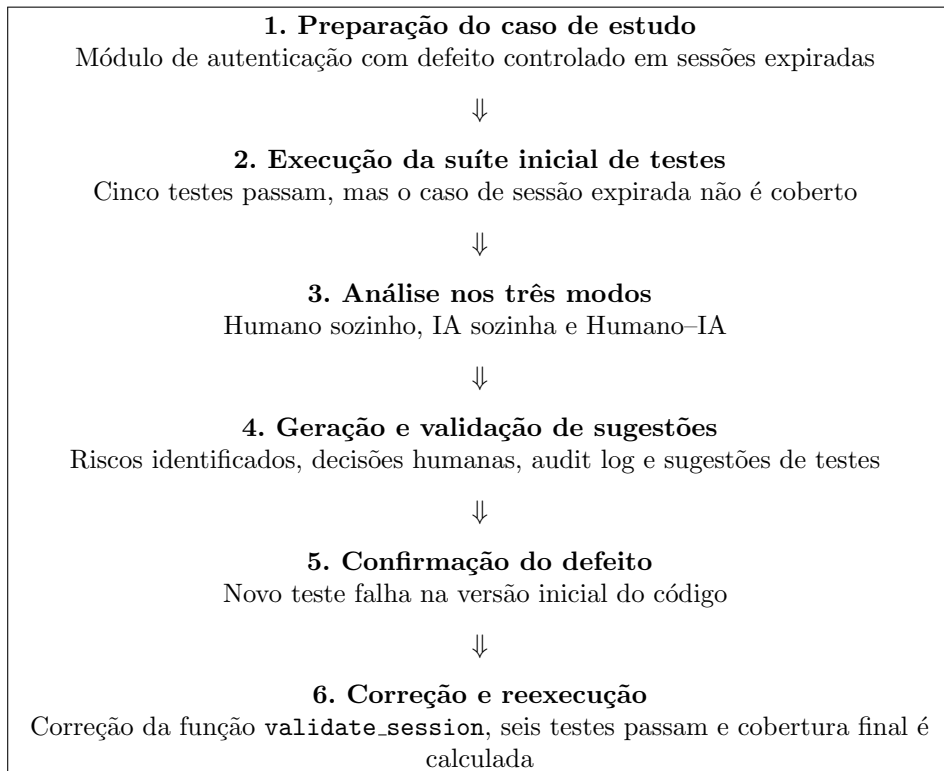


Fig. 4 Fluxo da metodologia experimental.

riscos relevantes, validar sugestões e documentar decisões. Por isso, a metodologia não se limita a contar testes ou medir cobertura. Ela inclui também métricas relacionadas com rastreabilidade e accountability.

6.2 Modos de Avaliação

Foram avaliados três modos de trabalho. Cada modo representa uma forma diferente de executar atividades de qualidade sobre o mesmo módulo de autenticação. A Tabela 8 descreve estes modos.

No modo humano sozinho, a revisão foi considerada como uma inspeção manual ao módulo de autenticação. O revisor poderia analisar a função `validate_session`, verificar os testes existentes e identificar manualmente a ausência de cobertura para sessões expiradas. Este modo é importante porque representa a prática tradicional de revisão humana, mas apresenta uma limitação: o processo não gera automaticamente um registo estruturado das decisões tomadas.

No modo IA sozinha, a análise foi realizada pelo módulo `ai_reviewer.py`. Este módulo identificou três riscos relevantes. No entanto, como não existe validação humana neste modo, as sugestões não podem ser consideradas decisões finais. Este modo permite mostrar que a IA pode aumentar a capacidade de análise, mas não

Table 8 Modos de avaliação considerados

Modo	Descrição
Humano sozinho	O código é analisado manualmente, sem utilização das sugestões do CollabQA. Este modo representa uma revisão tradicional dependente apenas do conhecimento e atenção do revisor.
IA sozinha	O módulo <code>ai_reviewer.py</code> gera sugestões automaticamente, sem validação humana. Este modo permite observar o que a análise automatizada consegue identificar, mas sem decisão final responsável.
Humano-IA	A IA gera sugestões, o humano valida cada uma delas, as decisões são registradas e os testes são sugeridos com base nos riscos aceites ou parcialmente aceites. Este é o fluxo completo do CollabQA.

resolve sozinha o problema da responsabilidade. Uma sugestão sem validação continua a ser apenas uma hipótese.

No modo Humano-IA, foi executado o fluxo completo do CollabQA. A IA gerou sugestões, o humano validou cada uma delas, o audit log registou as decisões e o módulo de geração de testes produziu testes associados aos riscos validados. Este modo representa a abordagem proposta pelo projeto e permite avaliar não apenas a quantidade de problemas encontrados, mas também a existência de justificação humana e rastreabilidade.

6.3 Métricas de Avaliação

A avaliação utilizou métricas quantitativas e qualitativas. As métricas quantitativas permitem comparar os modos de forma objetiva, enquanto as métricas qualitativas ajudam a interpretar o significado dos resultados. A Tabela 9 apresenta as métricas usadas.

Para alinhar a avaliação com os dados propostos no enunciado, foram também considerados indicadores relacionados com aceitação de sugestões, erros detetados ou não detetados, confiança e limitações da medição temporal. A taxa de aceitação foi observada através das decisões humanas registadas: duas sugestões foram aceites, uma foi aceite parcialmente e nenhuma foi rejeitada. A relação entre erros detetados e erros inicialmente não detetados foi analisada através do caso da sessão expirada, que não era coberto pelos cinco testes iniciais, mas foi posteriormente confirmado pelo teste `test_expired_session_is_invalid`. A confiança foi analisada qualitativamente através do questionário estruturado de reflexão, considerando se a confiança nas sugestões aumentava quando existia evidência executável. O tempo de decisão não foi medido formalmente com cronómetro, sendo tratado como uma limitação da experiência.

Table 9 Métricas usadas na avaliação experimental

Métrica	Descrição
Problemas relevantes encontrados	Número de riscos ou defeitos relevantes identificados durante a análise.
Falsos positivos	Número de sugestões consideradas incorretas ou sem relevância para o caso de estudo.
Testes úteis sugeridos	Número de testes propostos que contribuem para verificar riscos reais ou decisões de qualidade.
Decisões humanas validadas	Número de sugestões avaliadas por um humano, com decisão explícita.
Entradas no audit log	Número de decisões registadas com responsável, justificação, risco e categoria.
Resultado dos testes antes da correção	Número de testes que passaram e falharam quando o novo teste foi executado contra a versão inicial.
Resultado dos testes após a correção	Número de testes que passaram depois da correção da função <code>validate_session</code> .
Cobertura final	Percentagem de cobertura obtida após a correção e execução da suíte final de testes.

Table 10 Dados adicionais considerados em relação ao enunciado

Dado pedido no enunciado	Tratamento no CollabQA
Taxa de aceitação/rejeição	Foram registadas duas sugestões aceites, uma parcialmente aceite e zero rejeitadas.
Tempo de decisão	Não foi medido formalmente; esta ausência foi assumida como limitação metodológica.
Erros detetados vs. não detetados	O erro da sessão expirada não foi detetado pelos testes iniciais, mas foi identificado pela revisão assistida e confirmado por novo teste.
Confiança e confiança declarada	Foram analisadas qualitativamente no questionário estruturado de reflexão.
Logs de decisão e verificação	Foram registados em <code>decisions.json</code> , <code>audit_log.csv</code> e nos ficheiros da pasta <code>reports/</code> .
Questionário de reflexão	Foi incluído como artefacto complementar em <code>reports/reflection_questionnaire.md</code> e sintetizado na discussão.

A métrica “problemas relevantes encontrados” avalia a capacidade de cada modo em identificar riscos importantes no código. No caso de estudo, foram considerados relevantes os seguintes problemas: aceitação de sessões expiradas, atribuição automática de permissões ao administrador e previsibilidade dos tokens de sessão. A métrica “falsos positivos” foi incluída para penalizar modos que gerassem sugestões incorretas ou irrelevantes. Embora no caso avaliado não tenham sido registados falsos positivos, a métrica foi mantida para tornar a comparação mais completa.

A métrica “testes úteis sugeridos” avalia a capacidade de transformar problemas identificados em testes concretos. Esta métrica é particularmente importante porque o projeto está relacionado com qualidade de software e *Continuous Testing*. Uma

sugestão só se torna realmente útil quando pode ser convertida em evidência verificável. No caso principal, a sugestão sobre sessões expiradas originou um teste que confirmou o defeito.

As métricas “decisões humanas validadas” e “entradas no audit log” foram incluídas para avaliar a dimensão de accountability. Estas métricas distinguem o modo Humano-IA do modo IA sozinha. Ambos podem identificar problemas, mas apenas o modo Humano-IA associa as sugestões a decisões humanas justificadas e registradas. Esta diferença é essencial para a ligação ao seminário, onde foi defendido que a responsabilidade deve ser distribuída, mas nunca indefinida.

6.4 Procedimento Experimental

O procedimento experimental foi realizado em várias etapas. A Tabela 11 resume a sequência seguida.

Table 11 Etapas executadas na metodologia experimental

Etapa	Descrição
1	Implementação do módulo de autenticação com o defeito controlado na função <code>validate_session</code> .
2	Criação de cinco testes iniciais que validavam comportamentos básicos, mas não cobriam sessões expiradas.
3	Execução da suíte inicial de testes, confirmando que os cinco testes passavam.
4	Execução da análise assistida por IA através do módulo <code>ai_reviewer.py</code> .
5	Validação humana das sugestões geradas, com registo das decisões em <code>decisions.json</code> e <code>audit_log.csv</code> .
6	Geração de sugestões de testes com base nas decisões humanas.
7	Adição do teste <code>test_expired_session_is_invalid</code> .
8	Execução dos testes contra a versão inicial, observando uma falha no novo teste.
9	Correção da função <code>validate_session</code> .
10	Reexecução dos testes, confirmando que os seis testes passaram.
11	Medição da cobertura final com <code>coverage.py</code> .
12	Comparação entre humano sozinho, IA sozinha e Humano-IA.

Este procedimento permitiu demonstrar o ciclo completo de descoberta e correção do defeito. A etapa mais importante foi a adição do teste para sessões expiradas. Antes desse teste, a suíte existente passava com sucesso. Depois da sua adição, a implementação inicial falhou. Esta falha foi uma evidência objetiva de que o defeito existia. Após a correção, a mesma suíte passou com sucesso, mostrando que o comportamento esperado foi restaurado.

6.5 Avaliação da Falha e da Correção

A avaliação da falha foi feita através do teste `test_expired_session_is_invalid`. Este teste cria uma sessão válida, altera manualmente a sua data de expiração para

o passado e verifica se a função `validate_session` devolve `False`. Na versão inicial, o teste falhou porque a função devolvia `True`. Isto aconteceu porque a função apenas verificava a existência do token.

A saída da execução inicial indicou um teste falhado e cinco testes bem-sucedidos. Este resultado foi guardado no ficheiro `reports/failing_test_output.txt`. Depois da correção da função `validate_session`, a suíte foi executada novamente e os seis testes passaram. Esta saída foi guardada no ficheiro `reports/passing_test_output.txt`. A cobertura final foi guardada em `reports/coverage_after_fix.txt`.

Esta abordagem tem uma vantagem importante: a melhoria não é apenas descrita verbalmente, mas demonstrada através de evidência executável. O teste falha antes da correção e passa depois da correção. Esta sequência mostra que a colaboração Humano-IA não se limitou a produzir uma sugestão; originou um teste capaz de confirmar o defeito e validar a correção.

6.6 Sistema de Pontuação

Para comparar os três modos de trabalho, foi usado um sistema de pontuação simples. A pontuação não pretende representar uma métrica universal de qualidade, mas sim uma forma prática de sintetizar os resultados do caso de estudo. O sistema atribui peso positivo aos problemas relevantes encontrados, testes úteis sugeridos, decisões humanas validadas e entradas no audit log. Os falsos positivos recebem peso negativo.

A pontuação foi calculada pela Equação 1.

$$Score = 3P + 2T + 2D + 2A - 2F \quad (1)$$

Nesta equação, P representa o número de problemas relevantes encontrados, T representa o número de testes úteis sugeridos, D representa o número de decisões humanas validadas, A representa o número de entradas no audit log e F representa o número de falsos positivos.

A escolha dos pesos reflete as prioridades do projeto. Os problemas relevantes encontrados recebem o peso mais elevado, porque identificar riscos reais é essencial para a qualidade de software. Os testes úteis, decisões humanas e entradas no audit log recebem peso positivo significativo, porque representam validação, evidência e rastreabilidade. Os falsos positivos são penalizados, porque podem desperdiçar tempo da equipa e reduzir a confiança no processo.

6.7 Reprodutibilidade

A experiência foi desenhada para ser reprodutível. Todos os comandos principais foram registados, os artefactos foram guardados em ficheiros e os testes podem ser executados novamente através do terminal ou do workflow de GitHub Actions. A análise assistida por IA foi implementada de forma determinística, o que significa que a mesma entrada produz sempre as mesmas sugestões. Esta decisão reduz variabilidade e torna a avaliação mais controlada.

Os principais artefactos usados na avaliação foram os seguintes:

- `data/ai_suggestions.json`, com as sugestões da IA;

- `data/decisions.json`, com as decisões humanas;
- `data/audit_log.csv`, com o registo auditável das decisões;
- `data/test_suggestions.json`, com os testes sugeridos;
- `reports/failing_test_output.txt`, com a execução em que o novo teste falhou;
- `reports/passing_test_output.txt`, com a execução após a correção;
- `reports/coverage_after_fix.txt`, com a cobertura final;
- `reports/experiment_comparison.csv`, com a comparação entre modos.

Além disso, o workflow de GitHub Actions permite verificar automaticamente se os testes continuam a passar após alterações no repositório. Esta integração reforça a componente de *Continuous Testing* do projeto e fornece uma evidência adicional de que o sistema mantém o comportamento esperado.

6.8 Natureza da Avaliação

A avaliação deve ser interpretada como uma experiência controlada de protótipo. O caso de estudo é pequeno e foi desenhado especificamente para demonstrar o problema de sessões expiradas. Por isso, os resultados não devem ser generalizados diretamente para sistemas industriais complexos. Ainda assim, a metodologia é adequada ao objetivo do projeto, porque permite demonstrar, de forma concreta, como a colaboração Humano-IA pode apoiar a identificação de riscos, a criação de testes e o registo de responsabilidade.

A principal força da metodologia está na ligação entre sugestão, validação e evidência. A IA identifica um risco, o humano valida a sugestão, o sistema propõe um teste, o teste confirma o defeito, a correção é implementada e a nova execução dos testes confirma a melhoria. Este ciclo permite avaliar a colaboração Humano-IA de forma mais completa do que uma simples análise textual de sugestões.

7 Resultados

Esta secção apresenta os resultados obtidos com a execução do CollabQA sobre o caso de estudo do módulo de autenticação. Os resultados são organizados em cinco partes: sugestões produzidas pela camada de assistência por IA, decisões humanas registadas, sugestões de testes geradas, confirmação e correção do defeito, e comparação experimental entre os três modos de trabalho avaliados.

O resultado mais importante do projeto foi a confirmação de que a suíte inicial de testes dava uma falsa sensação de qualidade. Antes da análise assistida por IA, existiam cinco testes automatizados e todos passavam. No entanto, nenhum desses testes verificava o comportamento do sistema perante sessões expiradas. Após a sugestão da IA e a validação humana, foi adicionado um teste específico para esse caso. Esse teste falhou na implementação inicial, confirmando a existência do defeito. Depois da correção da função `validate_session`, todos os seis testes passaram e a cobertura final atingiu 93%.

A Figura 5 apresenta uma síntese do ciclo observado durante a execução do projeto.

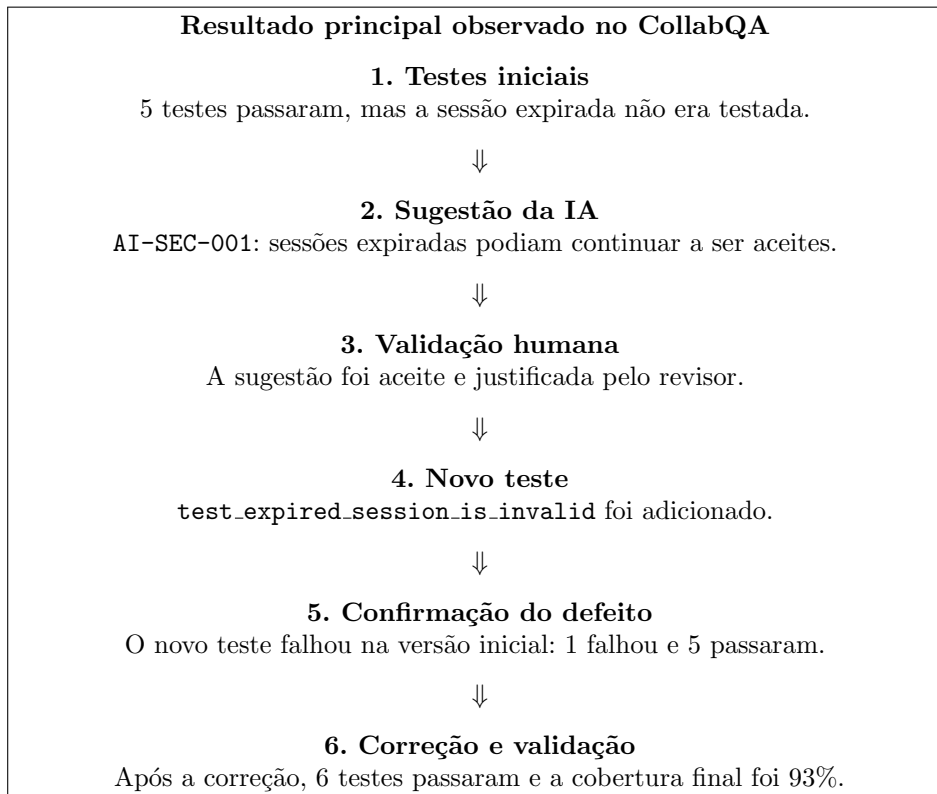


Fig. 5 Síntese do ciclo de descoberta, validação e correção do defeito.

7.1 Sugestões Produzidas pela IA

A primeira execução relevante do CollabQA correspondeu à análise do ficheiro `sample_project/auth.py` pelo módulo `app/ai_reviewer.py`. Esta análise produziu três sugestões. Cada sugestão foi identificada por um código, uma categoria, um nível de risco, uma descrição e uma recomendação. Todas as sugestões foram marcadas como exigindo validação humana.

A Tabela 12 apresenta as sugestões geradas pela camada de assistência por IA e as decisões humanas associadas.

A sugestão mais importante foi AI-SEC-001, porque corresponde diretamente ao defeito principal do caso de estudo. A análise identificou que a função `validate_session` apenas verificava se o token existia na estrutura de sessões, sem confirmar se a sessão já tinha expirado. Esta sugestão foi classificada como risco alto, uma vez que afeta segurança, fiabilidade e correção funcional.

A sugestão AI-AUTH-002 foi classificada como risco médio. Neste caso, a IA identificou que o utilizador administrador recebia permissões automaticamente. A decisão humana foi aceitar parcialmente esta sugestão, porque este comportamento poderia ser intencional num sistema simples, mas deveria ser documentado e revisto do ponto

Table 12 Sugestões geradas pela IA e decisões humanas

ID	Categoria	Risco	Problema identificado	Decisão humana
AI-SEC-001	Segurança	Alto	Sessões expiradas podiam continuar a ser aceites.	Aceite
AI-AUTH-002	Autorização	Médio	O administrador recebia permissões automaticamente.	Aceite parcial-
AI-REL-003	Fiabilidade	Médio	O token de sessão era previsível.	Aceite

de vista da segurança. Esta decisão é relevante porque demonstra que o humano não aceitou todas as sugestões de forma acrítica.

A sugestão **AI-REL-003** também foi classificada como risco médio. A IA identificou que o token de sessão era gerado de forma previsível, a partir do nome do utilizador. Esta sugestão foi aceite, uma vez que tokens previsíveis reduzem a segurança e podem causar problemas caso existam múltiplas sessões para o mesmo utilizador.

7.2 Decisões Humanas e Rastreabilidade

Todas as três sugestões foram avaliadas pelo revisor humano. Duas foram aceites e uma foi aceite parcialmente. Nenhuma sugestão foi rejeitada. O elemento mais importante destes resultados não é apenas a decisão final, mas o facto de cada decisão ter sido acompanhada por uma justificação e registada no audit log.

O audit log final contém três entradas, uma por cada sugestão avaliada. Cada entrada inclui identificador da sugestão, categoria, risco, título, decisão humana, justificação, responsável e indicação de que a validação humana era obrigatória. Isto demonstra que o processo não se limitou a gerar sugestões automáticas; houve uma etapa explícita de validação e responsabilização.

Este resultado é particularmente importante quando comparado com o modo IA sozinha. No modo IA sozinha, as três sugestões também foram produzidas, mas nenhuma delas foi validada por um humano e nenhuma decisão foi registada. Assim, a IA sozinha consegue apoiar a identificação de riscos, mas não fornece accountability suficiente para justificar a integração de uma alteração no processo de qualidade.

7.3 Resultados dos Testes

Antes da introdução do teste sugerido, a suíte inicial era composta por cinco testes. Todos passavam. No entanto, esses testes não cobriam o caso de sessão expirada. Depois da validação humana da sugestão **AI-SEC-001**, foi criado o teste `test_expired_session_is_invalid`. Este teste simulava uma sessão cuja data de expiração já tinha passado e verificava se a função `validate_session` devolvia `False`.

Quando o novo teste foi executado contra a versão inicial do módulo, o resultado foi uma falha. A função devolvia `True`, embora a sessão estivesse expirada. Este resultado

	timestamp	suggestion_id	category	risk	title	human_decision	justification	responsible	human_xa
0	2026-06-06T02:08:19	AI-SEC-001	Security	High	Expired sessions may still be accepted	Accepted	A sugestão foi aceita porque a função validate_session não verifica a expiração da ses	Guilherme Vicente	
1	2026-06-06T02:08:43	AI-AUTH-002	Authorization	Medium	Admin user receives permissions automatically	Partially accepted	A sugestão foi parcialmente aceita porque o comportamento do administrador pode	Guilherme Vicente	
2	2026-06-06T02:09:55	AI-REL-003	Reliability	Medium	Session tokens is predictable	Accepted	A sugestão foi aceita porque tokens previsíveis reduzem a segurança e podem causar	Guilherme Vicente	

Fig. 6 Audit log do CollabQA com as decisões humanas registadas, incluindo identificador da sugestão, categoria, risco, decisão, justificação e responsável.

confirmou que a falha descrita pela IA era real e tinha impacto no comportamento do sistema. Após a correção da função `validate_session`, todos os seis testes passaram.

A Tabela 13 apresenta a evolução dos resultados dos testes.

Table 13 Evolução dos resultados dos testes

Momento	Testes passados	Testes falhados	Interpretação
Suíte inicial	5	0	Os testes existentes passavam, mas não cobriam sessão expirada.
Após novo teste	5	1	O teste da sessão expirada falhou e confirmou o defeito.
Após correção	6	0	A função foi corrigida e todos os testes passaram.

Este resultado é um dos principais contributos do projeto. Ele mostra que a colaboração Humano-IA não produziu apenas uma recomendação textual. Produziu um teste concreto, esse teste confirmou um defeito e a correção foi validada por nova execução da suíte de testes. Assim, a qualidade melhorou com base em evidência executável.

```

sample_project/tests/test_auth.py:48: AssertionError
===== short test summary info =====
FAILED sample_project/tests/test_auth.py::test_expired_session_is_invalid - A...
===== 1 failed, 5 passed in 0.03s =====

```

Fig. 7 Execução do teste de sessão expirada antes da correção. O novo teste falhou porque a função `validate_session` aceitava uma sessão expirada como válida.

```

===== 6 passed in 0.02s =====
(venv) (base) MacBook-Air-de-Guilherme-2:CollabQA guilhermeficente$ pytest
===== test session starts =====
platform darwin -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/guilhermeficente/Desktop/Secretária - MacBook Air de Guilherme/UBI/Mestrado/QS/CollabQA
configfile: pytest.ini
testpaths: sample_project/tests
plugins: anyio-4.13.0
collected 6 items

sample_project/tests/test_auth.py ..... [100%]

===== 6 passed in 0.01s =====
(venv) (base) MacBook-Air-de-Guilherme-2:CollabQA guilhermeficente$ coverage run -m pytest
coverage report
===== test session starts =====
platform darwin -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/guilhermeficente/Desktop/Secretária - MacBook Air de Guilherme/UBI/Mestrado/QS/CollabQA
configfile: pytest.ini
testpaths: sample_project/tests
plugins: anyio-4.13.0
collected 6 items

sample_project/tests/test_auth.py ..... [100%]

===== 6 passed in 0.01s =====
(venv) (base) MacBook-Air-de-Guilherme-2:CollabQA guilhermeficente$ coverage report
Name                               Stmts   Miss  Cover
-----
sample_project/_init_.py             0      0  100%
sample_project/auth.py               31      4   87%
sample_project/tests/_init_.py        0      0  100%
sample_project/tests/test_auth.py     23      0  100%
TOTAL                               54      4   93%

```

Fig. 8 Execução final dos testes e relatório de cobertura após a correção da função `validate_session`. A suite final passou com seis testes e a cobertura total atingiu 93%.

7.4 Cobertura de Testes

Após a correção da função `validate_session`, a cobertura foi medida com `coverage.py`. O resultado final foi uma cobertura total de 93%. O ficheiro principal do caso de estudo, `sample_project/auth.py`, obteve 87% de cobertura, enquanto o ficheiro de testes apresentou 100% de cobertura no relatório de execução. A cobertura total considerada para a avaliação foi 93%.

A Tabela 14 apresenta o resumo da cobertura final.

Table 14 Cobertura final após a correção

Ficheiro	Linhas	Linhas não cobertas	Cobertura
<code>sample_project/auth.py</code>	31	4	87%
<code>sample_project/tests/test_auth.py</code>	23	0	100%
Total	54	4	93%

A cobertura elevada não deve ser interpretada como garantia absoluta de ausência de defeitos. Pelo contrário, o próprio caso de estudo demonstra que os testes iniciais podiam passar e ainda assim não cobrir um cenário crítico. No entanto, depois da

adição do teste de sessão expirada, a cobertura final passou a estar associada a uma suíte de testes mais relevante, porque passou a incluir o comportamento que originava o defeito principal.

7.5 Métricas Finais do CollabQA

O módulo `app/metrics.py` consolidou as métricas finais do projeto. Estas métricas incluem o número de sugestões geradas pela IA, decisões humanas, sugestões aceites, sugestões parcialmente aceites, sugestões de testes, entradas no audit log, resultados dos testes antes e depois da correção e cobertura final.

A Tabela 15 apresenta as principais métricas finais.

Table 15 Métricas finais do CollabQA

Métrica	Valor
Sugestões geradas pela IA	3
Sugestões validadas por humano	3
Sugestões aceites	2
Sugestões parcialmente aceites	1
Sugestões rejeitadas	0
Sugestões de alto risco	1
Sugestões de testes	3
Sugestões de testes de prioridade alta	1
Entradas no audit log	3
Testes falhados antes da correção	1
Testes passados antes da correção	5
Testes falhados após a correção	0
Testes passados após a correção	6
Cobertura final	93%

Estes resultados mostram que o processo completo produziu artefactos em todas as etapas esperadas: sugestões da IA, validação humana, audit log, testes sugeridos, confirmação do defeito, correção e métricas finais. O valor de 3 entradas no audit log é particularmente relevante, porque demonstra que cada sugestão teve uma decisão humana registada.

7.6 Comparação entre Modos de Trabalho

A avaliação comparou três modos de trabalho: humano sozinho, IA sozinha e Humano-IA. A Tabela 16 apresenta os resultados dessa comparação.

O modo humano sozinho identificou um problema relevante e sugeriu um teste útil, mas não produziu audit log nem rastreabilidade estruturada. Este modo representa uma revisão manual simples: pode encontrar problemas, mas depende fortemente da atenção do revisor e não gera automaticamente evidência de decisão.

O modo IA sozinha identificou três problemas relevantes e sugeriu três testes úteis. No entanto, não incluiu validação humana nem registo de responsabilidade. Por isso, apesar de ter melhor desempenho do que o humano sozinho em termos de quantidade

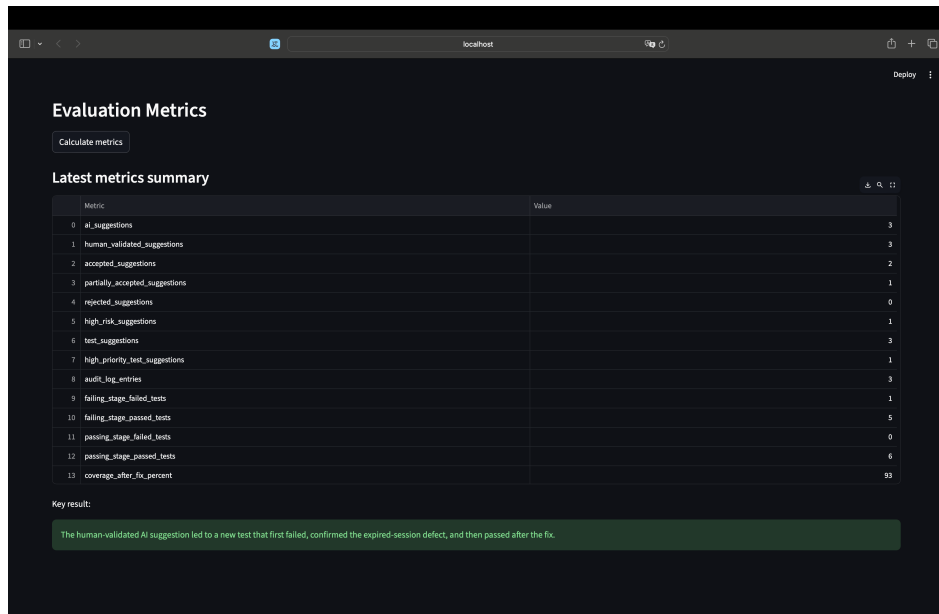


Fig. 9 Métricas finais apresentadas na interface do CollabQA, incluindo sugestões geradas, decisões humanas, entradas no audit log, resultados dos testes e cobertura final.

Table 16 Comparação experimental entre modos de trabalho

Modo	Problemas encontrados	Falsos positivos	Testes úteis	Decisões humanas	Audit log	Score
Humano sozinho	1	0	1	1	0	7
IA sozinha	3	0	3	0	0	15
Humano-IA	3	0	3	3	3	27

de problemas encontrados, não resolve a questão da accountability. As sugestões da IA continuam a ser hipóteses sem decisão validada.

O modo Humano-IA obteve o melhor resultado, com pontuação 27. Este modo identificou os três problemas relevantes, sugeriu três testes úteis, validou três decisões humanas e registou três entradas no audit log. Este resultado apoia a hipótese do projeto: a colaboração Humano-IA oferece um melhor equilíbrio entre capacidade de análise, validação, evidência e rastreabilidade.

7.7 Resposta às Questões de Investigação

A avaliação realizada permite responder às três questões de investigação associadas ao tema do projeto.

Relativamente à primeira questão, “*How does verification behavior differ between HITL and HOTL?*”, observou-se que o modo mais próximo de HITL promoveu uma verificação mais detalhada de cada sugestão. Cada recomendação exigiu uma decisão humana explícita e uma justificação, o que levou a uma análise contextual mais cuidadosa. Em contraste, a condição de supervisão reduzida, próxima da lógica HOTL, permitiu observar resultados globais através de métricas e audit log, mas não garantiu validação individual de cada sugestão antes da sua utilização. Assim, o HITL favoreceu precisão e controlo local, enquanto o HOTL favoreceu supervisão global e rastreabilidade agregada.

Relativamente à segunda questão, “*What types of AI errors are missed under reduced human oversight?*”, o caso de estudo mostrou que erros de lógica de segurança podem permanecer escondidos quando não existe verificação adequada. No caso analisado, a função `validate_session` aceitava tokens existentes sem verificar a data de expiração. Os testes iniciais passavam, mas o cenário crítico de sessão expirada não era coberto. Assim, sem validação humana e sem o teste específico sugerido, este defeito poderia continuar a gerar uma falsa sensação de segurança.

Relativamente à terceira questão, “*How does trust correlate with decision accuracy?*”, os resultados sugerem que a confiança deve depender da existência de evidência. A sugestão **AI-SEC-001** tornou-se fiável porque foi validada por um humano e confirmada por um teste automatizado que falhou antes da correção e passou depois. Por outro lado, a sugestão **AI-AUTH-002** foi apenas parcialmente aceite, demonstrando que a confiança não deve ser automática. Este comportamento traduz a ideia de confiança calibrada: aceitar sugestões quando há evidência suficiente e manter análise crítica quando a sugestão depende de contexto.

7.8 Interpretação dos Resultados

A interpretação principal dos resultados é que a IA tem valor quando usada como apoio, mas esse valor aumenta quando existe validação humana. A IA sozinha conseguiu identificar mais problemas do que o humano sozinho no caso de estudo, mas não produziu responsabilidade explícita. Por outro lado, o humano sozinho produziu julgamento contextual, mas encontrou menos problemas e não deixou um registo estruturado. O modo Humano-IA combinou as vantagens dos dois.

A sugestão **AI-SEC-001** demonstra esta combinação de forma clara. A IA identificou um risco real. O humano validou a sugestão e justificou a decisão. O sistema gerou um teste associado ao risco. O teste falhou na versão inicial, confirmando o defeito. Após a correção, os testes passaram. Este ciclo mostra que a colaboração Humano-IA pode produzir melhoria real de qualidade quando está associada a testes e evidência.

Outro resultado importante foi a decisão parcialmente aceite em **AI-AUTH-002**. Esta decisão mostra que o humano não funcionou apenas como confirmador automático das sugestões da IA. Pelo contrário, avaliou o contexto e concluiu que o comportamento podia ser intencional, embora devesse ser documentado e revisto. Isto reforça a ideia de confiança calibrada: nem aceitação cega nem rejeição automática.

A ausência de falsos positivos deve ser interpretada com cuidado. Como o caso de estudo foi controlado e a camada de IA foi determinística, todas as sugestões geradas foram consideradas relevantes. Num cenário real com um modelo de linguagem

externo, poderiam surgir sugestões incorretas, redundantes ou pouco úteis. Ainda assim, o protótipo inclui a métrica de falsos positivos precisamente para permitir essa análise em versões futuras.

Em síntese, os resultados demonstram que o CollabQA conseguiu transformar os princípios do seminário numa prática observável. O projeto mostrou que uma sugestão da IA pode originar uma decisão humana, um teste, a confirmação de um defeito, uma correção e uma evidência de qualidade. Esta cadeia é mais forte do que uma utilização informal da IA, porque torna explícito o percurso entre recomendação automática e validação humana.

8 Discussão

Os resultados obtidos permitem discutir o papel da colaboração Humano-IA em atividades de qualidade de software a partir de uma situação concreta: um módulo de autenticação com uma falha controlada na validação de sessões expiradas. O principal contributo do projeto não está apenas no facto de ter sido identificado e corrigido um defeito, mas na forma como esse defeito foi descoberto, validado, documentado e transformado em evidência através de testes automatizados.

A primeira conclusão relevante é que a existência de testes a passar não é suficiente para garantir qualidade. Na fase inicial do projeto, os cinco testes automatizados passavam com sucesso. À primeira vista, este resultado poderia sugerir que o módulo de autenticação estava correto. No entanto, os testes não cobriam o cenário de sessão expirada, que era precisamente o caso crítico do estudo. Isto mostra que a qualidade da suíte de testes não depende apenas do número de testes ou do facto de todos passarem, mas também da relevância dos cenários cobertos.

Este ponto é particularmente importante em sistemas com requisitos de segurança. Um módulo de autenticação pode parecer funcional se o login, o logout e as permissões básicas forem testados. Contudo, comportamentos como expiração de sessão, reutilização de tokens, invalidação de credenciais e permissões especiais são igualmente importantes. No caso estudado, a função `validate_session` aceitava qualquer token existente, mesmo que a sessão já tivesse expirado. Assim, o defeito não estava na ausência total de testes, mas na ausência de um teste para um cenário crítico.

A análise assistida por IA contribuiu para tornar esse risco visível. O módulo `ai_reviewer.py` identificou a sugestão AI-SEC-001, classificando-a como risco alto. Esta sugestão apontava diretamente para a falha na validação de sessões expiradas. O valor da IA, neste caso, esteve na sua capacidade de chamar a atenção para um risco que não estava coberto pela suíte inicial de testes. No entanto, a sugestão por si só não seria suficiente para garantir qualidade. Uma recomendação automática continua a ser apenas uma hipótese enquanto não for validada.

É neste ponto que a decisão humana se torna essencial. O humano analisou a sugestão, aceitou-a e justificou a decisão. Esta etapa distingue o CollabQA de uma ferramenta automática simples. A IA não corrigiu diretamente o código, não aprovou uma alteração e não decidiu sozinha que o sistema estava incorreto. A sua função foi apoiar a análise. A decisão final pertenceu ao humano, que avaliou o risco e confirmou

que a sugestão era relevante. Isto reforça a ideia de confiança calibrada: a IA não deve ser ignorada, mas também não deve ser aceite de forma acrítica.

A decisão parcialmente aceite em **AI-AUTH-002** é um resultado importante para esta discussão. Se todas as sugestões tivessem sido aceites sem distinção, o projeto poderia parecer apenas uma validação automática das respostas da IA. No entanto, o comportamento do administrador foi tratado de forma mais cuidadosa. A sugestão foi aceite parcialmente porque a atribuição automática de permissões ao administrador pode ser intencional num sistema simples, embora deva ser documentada e revista por razões de segurança. Esta decisão mostra que o humano continua a interpretar o contexto, distinguindo defeitos claros de decisões de desenho que exigem análise adicional.

A criação do teste `test_expired_session_is_invalid` foi o momento em que a sugestão deixou de ser apenas uma observação e passou a ser evidência executável. Este teste confirmou objetivamente o defeito: na versão inicial, a função devolvia **True** para uma sessão expirada. Após a correção, o mesmo teste passou. Esta sequência é fundamental para qualidade de software, porque transforma uma suspeita em prova e uma correção em comportamento verificado.

Esta interpretação também se relaciona com a importância do teste exploratório, onde o julgamento humano continua a ser essencial para observar comportamentos inesperados, adaptar a análise e interpretar o impacto real de uma falha [4].

Outro aspeto relevante é a diferença entre deteção de problemas e responsabilização por decisões. O modo IA sozinho identificou três problemas relevantes, tal como o modo Humano-IA. No entanto, no modo IA sozinho não existiram decisões humanas validadas nem entradas no audit log. Isto significa que, embora a IA tenha contribuído para a análise, não existia uma cadeia de responsabilidade. Num contexto real, esta diferença seria crítica. Uma equipa não deve integrar alterações ou assumir conclusões apenas porque uma ferramenta gerou uma recomendação. Deve existir uma pessoa ou equipa responsável pela validação.

A comparação experimental mostra precisamente essa diferença. O modo IA sozinho obteve melhor desempenho do que o humano sozinho em termos de problemas encontrados e testes sugeridos, mas falhou na dimensão de accountability. O modo humano sozinho forneceu julgamento humano, mas encontrou menos problemas e não gerou rastreabilidade estruturada. O modo Humano-IA combinou os pontos fortes dos dois: identificou três problemas, sugeriu três testes, registou três decisões humanas e criou três entradas no audit log. Por isso obteve a maior pontuação experimental.

A Tabela 17 resume a interpretação dos principais resultados.

A cobertura final de 93% deve ser interpretada com algum cuidado. Embora seja um valor positivo, o próprio projeto mostra que cobertura elevada não garante ausência de defeitos. Antes da introdução do teste de sessão expirada, o sistema já apresentava bons resultados nos testes existentes, mas continuava vulnerável ao defeito principal. Assim, a cobertura deve ser vista como uma métrica útil, mas incompleta. O mais importante é a combinação entre cobertura, qualidade dos cenários testados e ligação dos testes aos riscos identificados.

Table 17 Interpretação dos principais resultados

Resultado observado	Interpretação
Cinco testes iniciais passaram	A suíte inicial validava comportamentos básicos, mas não cobria o cenário crítico de sessão expirada.
A IA identificou três riscos	A análise automatizada aumentou a capacidade de detecção de problemas no código.
Duas sugestões foram aceites e uma parcialmente aceite	O humano avaliou o contexto e não aceitou todas as sugestões de forma cega.
O novo teste falhou antes da correção	A sugestão validada transformou-se em evidência executável e confirmou o defeito.
Seis testes passaram após a correção	A correção da função <code>validate_session</code> restaurou o comportamento esperado.
A cobertura final foi 93%	A suíte final apresentou boa cobertura, agora incluindo o cenário crítico identificado.
O audit log teve três entradas	As decisões ficaram rastreáveis, com responsável e justificação.
O modo Humano-IA obteve a maior pontuação	A colaboração combinou análise automatizada, julgamento humano e accountability.

O projeto também demonstra a importância de ligar revisão de código e testes. Muitas vezes, estas atividades são tratadas separadamente: a revisão identifica problemas e os testes verificam comportamento. No CollabQA, estas duas dimensões foram integradas. A IA identificou um risco na revisão de código, o humano validou esse risco e a ferramenta sugeriu um teste correspondente. Este fluxo cria uma ponte entre *Continuous Review* e *Continuous Testing*. A revisão não termina numa observação; gera uma ação verificável sob a forma de teste automatizado.

A utilização de GitHub Actions reforça esta ligação. Após a correção, os testes passaram localmente e também puderam ser executados automaticamente no repositório. Isto aproxima o projeto de um cenário real de desenvolvimento, onde cada alteração deve ser validada automaticamente antes de ser integrada. Embora o caso de estudo seja pequeno, o princípio é o mesmo usado em projetos maiores: automatizar validações repetíveis para reduzir o risco de regressões.

Do ponto de vista da responsabilidade, o audit log é um dos componentes mais relevantes. O registo das decisões permite responder a perguntas importantes: que sugestão foi feita, qual era o risco, quem avaliou, qual foi a decisão e por que razão essa decisão foi tomada. Sem este registo, seria difícil reconstruir o processo depois de uma falha. Com o audit log, a utilização da IA torna-se mais transparente e auditável.

Esta discussão também permite clarificar uma ideia importante: a IA não deve ser tratada como culpada nem como responsável final. No caso de uma falha, a IA pode estar na origem de uma sugestão incorreta ou incompleta, mas a integração dessa sugestão depende de decisões humanas e organizacionais. No CollabQA, esta ideia é refletida pelo facto de a IA apenas gerar sugestões. A decisão final pertence ao humano. Portanto, a responsabilidade não desaparece com a utilização de IA; pelo contrário, deve tornar-se mais explícita.

Ao mesmo tempo, também seria incorreto concluir que a IA não tem valor. No caso estudado, a análise assistida ajudou a identificar riscos relevantes e a propor

testes úteis. O problema não está em usar IA, mas em usá-la sem validação. A abordagem mais adequada não é rejeitar estas ferramentas nem confiar nelas cegamente. A abordagem mais produtiva é integrá-las num processo em que as recomendações são verificadas, documentadas e acompanhadas por testes.

A opção por uma camada de IA determinística tem vantagens e limitações. A principal vantagem é a reprodutibilidade. Como as sugestões são sempre iguais para o mesmo código, é possível repetir a experiência e comparar os resultados sem variabilidade. Esta escolha foi adequada para um projeto académico, onde o objetivo principal era demonstrar o processo de colaboração. No entanto, uma ferramenta baseada num modelo de linguagem real poderia produzir sugestões mais variadas, incluindo sugestões incorretas, redundantes ou inesperadas. Por isso, numa versão futura, seria importante testar o CollabQA com um modelo real e avaliar a robustez do processo perante respostas menos previsíveis.

Outra questão relevante é a escala do caso de estudo. O módulo de autenticação usado é pequeno e controlado. Esta simplicidade é positiva para demonstrar o conceito, mas limita a generalização dos resultados. Em sistemas reais, o código é mais complexo, os requisitos são mais numerosos, os efeitos colaterais são mais difíceis de prever e os testes podem depender de bases de dados, serviços externos ou interfaces. Ainda assim, o fluxo implementado no CollabQA é transferível: análise assistida, validação humana, sugestão de testes, audit log e execução automatizada podem ser aplicados a contextos maiores.

A comparação entre os três modos de trabalho também deve ser vista como uma avaliação exploratória e não como uma prova estatística. O objetivo foi comparar comportamentos num caso de estudo controlado, não medir desempenho humano ou de IA em larga escala. Mesmo assim, a comparação é útil porque torna visível a diferença entre encontrar problemas e construir um processo responsável de qualidade. O modo Humano-IA destacou-se porque não avaliou apenas quantidade de problemas encontrados, mas também validação e rastreabilidade.

O questionário estruturado de reflexão permitiu complementar a análise quantitativa com uma interpretação qualitativa do processo. A reflexão mostrou que a confiança nas sugestões automáticas foi maior quando a sugestão podia ser verificada por teste, como aconteceu com AI-SEC-001. Também evidenciou o risco de *over-reliance*: se todas as sugestões fossem aceites automaticamente, decisões de desenho poderiam ser tratadas como defeitos reais. O caso AI-AUTH-002 ilustra precisamente este ponto, uma vez que foi aceite apenas parcialmente por poder representar uma política intencional para o administrador. Por outro lado, a experiência mostrou também risco de *under-verification*, porque os cinco testes iniciais passavam e, mesmo assim, não cobriam a sessão expirada. Desta forma, a reflexão reforça que a confiança deve ser calibrada pela combinação entre sugestão automática, julgamento humano e evidência executável.

Do ponto de vista pedagógico, o projeto também é relevante porque mostra de forma prática conceitos que muitas vezes ficam abstratos. Termos como confiança calibrada, accountability, responsabilidade distribuída, validação humana e rastreabilidade tornam-se mais claros quando associados a artefactos concretos: sugestões JSON, decisões humanas, audit log, testes automatizados, cobertura e GitHub Actions. Esta concretização facilita a compreensão do papel da IA na qualidade de software.

Em síntese, os resultados indicam que a colaboração Humano-IA pode melhorar atividades de qualidade de software quando é integrada num processo controlado. A IA contribui com análise e sugestões. O humano contribui com julgamento contextual e responsabilidade. Os testes fornecem evidência executável. O audit log garante rastreabilidade. A integração contínua permite repetir validações. O valor do CollabQA está precisamente na combinação destes elementos.

9 Ameaças à Validade

Apesar de o CollabQA permitir demonstrar de forma prática a colaboração Humano-IA em atividades de qualidade de software, existem algumas ameaças à validade que devem ser consideradas. Estas ameaças não invalidam os resultados obtidos, mas delimitam o seu alcance e ajudam a interpretar corretamente as conclusões. Como o projeto foi desenvolvido em contexto académico e aplicado a um caso de estudo controlado, os resultados devem ser entendidos como evidência exploratória e não como prova definitiva aplicável a todos os contextos industriais.

Uma primeira ameaça está relacionada com a dimensão do caso de estudo. O módulo de autenticação utilizado é pequeno e contém um conjunto limitado de funções: login, logout, validação de sessão, verificação de permissões e limpeza de sessões. Esta simplicidade foi útil para isolar o problema principal e demonstrar claramente o fluxo proposto. No entanto, sistemas reais de autenticação são mais complexos, podendo envolver bases de dados, encriptação de palavras-passe, tokens aleatórios, refresh tokens, permissões hierárquicas, sessões distribuídas, integração com serviços externos e mecanismos de segurança adicionais. Assim, os resultados obtidos não devem ser generalizados diretamente para aplicações reais sem validação adicional.

Outra ameaça relevante está associada ao facto de o defeito ter sido controlado. A falha na validação de sessões expiradas foi introduzida intencionalmente para representar o caso de estudo discutido no seminário. Esta decisão permitiu criar uma experiência clara e verificável, mas também significa que o defeito era conhecido no desenho experimental. Em projetos reais, os defeitos surgem de forma menos previsível e podem estar distribuídos por vários componentes. Ainda assim, a utilização de um defeito controlado foi adequada para demonstrar o ciclo completo: identificação do risco, validação humana, geração de teste, confirmação da falha, correção e reexecução dos testes.

A componente de inteligência artificial (IA) constitui outra ameaça à validade. No protótipo implementado, a camada de IA foi simulada através de uma análise determinística baseada em padrões do código. Esta opção foi tomada para garantir reprodutibilidade experimental, uma vez que a mesma entrada produz sempre as mesmas sugestões. No entanto, esta abordagem não representa totalmente o comportamento de um modelo de linguagem real, que poderia gerar respostas diferentes, incompletas, redundantes ou incorretas. Por isso, os resultados demonstram o processo de colaboração Humano-IA, mas não avaliam diretamente o desempenho de um modelo real de IA.

Também existe uma ameaça relacionada com a comparação entre os três modos de trabalho. O modo humano sozinho, o modo IA sozinho e o modo Humano-IA foram

avaliados num cenário pequeno e controlado. A pontuação atribuída a cada modo foi calculada com base numa fórmula simples, definida para este projeto. Esta fórmula valoriza problemas encontrados, testes úteis, decisões humanas e audit log, penalizando falsos positivos. Embora seja coerente com os objetivos do trabalho, não deve ser interpretada como uma métrica universal de qualidade. Em contextos diferentes, os pesos poderiam ser ajustados de acordo com outros critérios, como criticidade do sistema, custo de revisão, tempo de execução ou impacto dos defeitos.

A avaliação do modo humano sozinho também pode ter sido influenciada pelo conhecimento prévio do projeto. Como o caso de estudo foi desenvolvido pela própria equipa, existe o risco de o revisor já conhecer parcialmente o defeito ou a intenção experimental. Isto pode afetar a comparação com situações reais, onde o programador ou tester pode não conhecer previamente os problemas existentes. Para mitigar esta limitação, o relatório apresenta a comparação como uma avaliação controlada e não como um estudo empírico com participantes independentes.

Outra limitação está relacionada com o número de participantes. O projeto não incluiu um estudo com várias equipas, vários programadores ou vários testers. A validação humana foi realizada no contexto da própria equipa do projeto. Assim, não é possível tirar conclusões estatísticas sobre o comportamento de diferentes perfis de utilizadores perante sugestões de IA. Um estudo mais abrangente poderia envolver vários participantes, medir tempo de revisão, analisar taxa de aceitação de sugestões e comparar a qualidade das decisões entre revisores com diferentes níveis de experiência.

Outra ameaça à validade está relacionada com a medição do tempo de decisão e da confiança declarada. O enunciado do projeto propunha a recolha de dados como *time-to-decision* e *self-reported trust and confidence*. Neste protótipo, essas dimensões foram analisadas de forma qualitativa através do questionário estruturado de reflexão, mas o tempo de decisão não foi medido formalmente com cronómetro nem foi aplicado um questionário quantitativo a vários participantes. Esta limitação reduz a força empírica da comparação, mas foi assumida explicitamente devido ao âmbito académico e controlado do trabalho.

A cobertura de testes também deve ser interpretada com cuidado. A cobertura final de 93% é um resultado positivo, mas não prova a ausência total de defeitos. O próprio projeto demonstra que testes podem passar e ainda assim não cobrir cenários críticos. Portanto, a cobertura foi usada como uma métrica auxiliar e não como garantia absoluta de qualidade. A melhoria mais relevante não foi apenas o aumento ou manutenção da cobertura, mas sim a introdução de um teste significativo para o cenário de sessão expirada.

A utilização de ficheiros JSON e CSV para guardar sugestões, decisões e audit log é suficiente para um protótipo académico, mas apresenta limitações em termos de robustez, concorrência e persistência. Num sistema real, estes dados deveriam provavelmente ser armazenados numa base de dados, associados a commits, pull requests, utilizadores autenticados e histórico de revisões. Ainda assim, para o objetivo do projeto, os ficheiros utilizados permitem demonstrar claramente a rastreabilidade das decisões.

A interface Streamlit também deve ser entendida como uma ferramenta de demonstração e não como uma aplicação final. A interface permite visualizar o fluxo completo

do CollabQA, mas não implementa autenticação de utilizadores, controlo de permissões, gestão avançada de projetos ou integração direta com plataformas de revisão de código. O seu objetivo principal foi tornar visível a sequência entre sugestão da IA, decisão humana, sugestão de teste, métricas e audit log.

Por fim, existe uma ameaça associada à generalização dos resultados para outros tipos de sistemas. O projeto foca-se num módulo de autenticação, que é uma área crítica e adequada para discutir segurança e responsabilidade. No entanto, a colaboração Humano-IA em qualidade de software pode assumir formas diferentes em sistemas de dados, aplicações móveis, sistemas distribuídos, interfaces gráficas, APIs complexas ou software crítico. A arquitetura do CollabQA foi desenhada para ser extensível, mas essa extensão não foi validada neste trabalho.

A Tabela 18 resume as principais ameaças à validade e as estratégias adotadas para as mitigar.

Table 18 Ameaças à validade e estratégias de mitigação

Ameaça à validade	Mitigação ou interpretação adotada
Caso de estudo pequeno	O módulo foi mantido pequeno para permitir controlo experimental e demonstração clara do fluxo proposto.
Defeito controlado	A falha foi introduzida intencionalmente para reproduzir o cenário do seminário e permitir validação objetiva.
IA simulada e determinística	A abordagem garante reprodutibilidade; versões futuras poderão integrar modelos de linguagem reais.
Fórmula de pontuação simples	A pontuação foi usada apenas como síntese exploratória, não como métrica universal de qualidade.
Conhecimento prévio do caso de estudo	Os resultados são apresentados como avaliação controlada, não como estudo empírico independente.
Número reduzido de participantes	O projeto avalia o processo e o protótipo, não o comportamento estatístico de diferentes utilizadores.
Cobertura não garante ausência de defeitos	A cobertura foi interpretada juntamente com a relevância dos cenários testados.
Persistência simples em JSON e CSV	A solução é adequada ao protótipo; uma versão industrial exigiria base de dados e integração com pull requests.
Interface de demonstração	O Streamlit foi usado para demonstrar o fluxo, não como produto final completo.
Generalização limitada	Os resultados aplicam-se diretamente ao caso de estudo e devem ser validados em projetos maiores.

Em síntese, as ameaças à validade mostram que o CollabQA deve ser interpretado como um protótipo académico e exploratório. O trabalho não pretende provar que a colaboração Humano-IA será sempre superior em qualquer contexto. O que demonstra é que, num caso de estudo controlado, esta colaboração permitiu identificar um risco real, validar humanamente a sugestão, gerar um teste útil, confirmar o defeito, corrigir o código e manter um registo auditável das decisões. Esta contribuição é coerente com os objetivos do projeto e fornece uma base sólida para evolução futura.

10 Conclusão

Este projeto apresentou o CollabQA, um protótipo desenvolvido para estudar a colaboração Humano-IA em atividades de qualidade de software, usando como caso de estudo um módulo de autenticação. O trabalho partiu do seminário sobre confiança, accountability e responsabilidade em engenharia de software apoiada por IA, procurando transformar a reflexão conceptual apresentada nesse seminário numa solução prática, executável e avaliável.

O problema central abordado foi a utilização de IA em tarefas de revisão de código, geração de testes e apoio à depuração sem perda de validação humana, rastreabilidade e responsabilidade. A IA pode apoiar programadores e testers, mas as suas sugestões não devem ser tratadas como decisões finais. Esta ideia orientou toda a implementação do CollabQA: a IA sugere, o humano valida, os testes fornecem evidência e o audit log regista a cadeia de decisão.

O caso de estudo escolhido foi um módulo de autenticação simples, contendo funcionalidades de login, logout, validação de sessão e verificação de permissões. A falha principal estava na função `validate_session`, que inicialmente apenas verificava se o token existia, sem confirmar se a sessão já tinha expirado. Este defeito foi escolhido por estar diretamente relacionado com o cenário discutido no seminário e por representar um problema relevante de segurança e correção funcional.

A implementação do projeto permitiu demonstrar um ciclo completo de qualidade. Primeiro, os testes iniciais foram executados e todos passaram, apesar de não cobrirem o cenário de sessão expirada. Em seguida, a camada de análise assistida por IA identificou três riscos no código: aceitação de sessões expiradas, atribuição automática de permissões ao administrador e previsibilidade dos tokens de sessão. Depois, o humano validou essas sugestões, aceitando duas e aceitando parcialmente uma. Cada decisão foi acompanhada por uma justificação e registada no audit log.

O resultado mais importante ocorreu quando a sugestão relacionada com sessões expiradas foi transformada num teste automatizado. O teste `test_expired_session_is_invalid` falhou na implementação inicial, confirmando que o defeito existia. Após a correção da função `validate_session`, todos os seis testes passaram. A cobertura final atingiu 93%, demonstrando que a suíte de testes passou a incluir o cenário crítico que anteriormente estava ausente.

A comparação experimental entre humano sozinho, IA sozinha e Humano-IA mostrou que o modo colaborativo obteve o melhor resultado. O humano sozinho identificou menos problemas e não produziu rastreabilidade estruturada. A IA sozinha identificou três riscos, mas não forneceu validação humana nem accountability. O modo Humano-IA combinou os pontos fortes dos dois: identificou os três riscos, gerou três sugestões de testes, validou três decisões humanas e registou três entradas no audit log. Por isso, obteve a maior pontuação experimental, com um score de 27.

A principal conclusão do projeto é que a IA pode contribuir para a qualidade de software quando é integrada num processo controlado. O seu valor não está em substituir o programador, o tester ou o revisor, mas em apoiar a identificação de riscos e a geração de hipóteses de teste. Essas hipóteses só se tornam realmente úteis quando são avaliadas por humanos, documentadas e confirmadas através de testes

automatizados. Assim, a colaboração Humano-IA deve ser vista como uma relação de apoio e validação, não como uma transferência de responsabilidade para a ferramenta.

O projeto também demonstrou a importância da rastreabilidade. Em processos de desenvolvimento apoiados por IA, é fundamental saber que sugestão foi produzida, quem a avaliou, qual foi a decisão tomada e por que razão essa decisão foi aceite, rejeitada ou aceite parcialmente. O audit log implementado no CollabQA permitiu registar essa informação e transformar a utilização da IA num processo auditável. Esta componente é particularmente relevante em contextos onde falhas de qualidade podem ter impacto em segurança, privacidade ou confiança dos utilizadores.

Apesar dos resultados positivos, o projeto apresenta limitações. O caso de estudo é pequeno e controlado, a camada de IA foi implementada de forma determinística e a avaliação foi realizada num contexto académico. Assim, os resultados não devem ser interpretados como prova de que a colaboração Humano-IA será sempre superior em qualquer projeto. No entanto, o trabalho demonstra de forma concreta que, no caso estudado, esta colaboração permitiu identificar um risco real, gerar um teste útil, confirmar um defeito, corrigir o código e manter registo das decisões.

Como trabalho futuro, o CollabQA poderia ser expandido em várias direções. A primeira melhoria seria integrar um modelo de linguagem real na camada de assistência por IA, permitindo avaliar sugestões mais variadas e próximas de uma utilização prática. A segunda melhoria seria aplicar o protótipo a projetos maiores, com diferentes módulos, múltiplos ficheiros e cenários de teste mais complexos. A terceira melhoria seria integrar o sistema diretamente com pull requests, de modo a associar sugestões da IA, decisões humanas e audit log a alterações reais no repositório. Outra evolução possível seria substituir os ficheiros JSON e CSV por uma base de dados, permitindo melhor gestão histórica das decisões.

Em síntese, o CollabQA mostra que a colaboração Humano-IA pode ser útil para a qualidade de software quando é acompanhada por validação humana, testes automatizados, integração contínua e registo auditável. O projeto confirma a ideia central defendida no seminário: a IA pode apoiar decisões de engenharia de software, mas a responsabilidade final deve permanecer humana, explícita e rastreável.

Appendix A Questionário Estruturado de Reflexão

Este apêndice apresenta o questionário estruturado de reflexão usado para complementar a avaliação experimental. O objetivo foi refletir sobre confiança, *over-reliance*, *under-verification*, diferenças entre HITL e HOTL, precisão das decisões e limitações da experiência.

Q1. Em que situações houve maior confiança nas sugestões automáticas?

A confiança foi maior na sugestão AI-SEC-001, porque estava diretamente ligada a uma falha objetiva na função `validate_session`. Esta sugestão pôde ser validada através de um teste automatizado.

Q2. Houve risco de *over-reliance*?

Sim. Se todas as sugestões fossem aceites automaticamente, decisões de desenho poderiam ser tratadas como defeitos reais. O caso **AI-AUTH-002** mostra isso, porque foi aceite apenas parcialmente devido à possibilidade de o comportamento do administrador ser intencional.

Q3. Houve risco de *under-verification*?

Sim. Antes da criação do teste da sessão expirada, os cinco testes iniciais passavam, mas o defeito continuava escondido. Isto mostra que a ausência de verificação específica pode criar falsa confiança na qualidade do sistema.

Q4. Que erro foi detetado graças à validação?

Foi detetado que `validate_session` aceitava tokens existentes sem verificar `expires_at`. O teste `test_expired_session_is_invalid` confirmou o defeito.

Q5. Como mudou a confiança depois da validação?

Antes da validação, a confiança era apenas aparente, porque os testes existentes passavam. Depois da validação humana e da criação do novo teste, a confiança aumentou porque passou a existir evidência executável.

Q6. Qual foi a diferença entre HITL e HOTL no protótipo?

No HITL, cada sugestão exigiu decisão humana antes de avançar. No HOTL, entendido neste projeto como supervisão global ou reduzida, o humano observou métricas e audit log, com menor intervenção direta em cada sugestão individual.

Q7. Que dados de decisão foram observados?

Foram geradas três sugestões. Duas foram aceites, uma foi aceite parcialmente e nenhuma foi rejeitada. Isto permitiu observar a taxa de aceitação e a necessidade de análise contextual.

Q8. Que relação existiu entre confiança e precisão?

A confiança aumentou quando as sugestões foram confirmadas por testes. A sugestão **AI-SEC-001** tornou-se precisa porque foi validada por um teste que primeiro falhou e depois passou após a correção.

Q9. O tempo de decisão foi medido formalmente?

Não. O tempo de decisão não foi medido com cronómetro, sendo esta uma limitação da experiência. Ainda assim, observou-se qualitativamente que a sugestão **AI-SEC-001** foi mais fácil de validar por estar ligada a um defeito objetivo, enquanto **AI-AUTH-002** exigiu mais análise contextual.

Q10. Qual foi a principal limitação da experiência?

A camada de assistência é determinística e baseada em regras, não um modelo de linguagem real. Isto aumenta a reprodutibilidade, mas limita a generalização. Outra limitação é o facto de a avaliação ter sido realizada num caso de estudo pequeno e controlado, sem vários participantes.

References

- [1] Wang, A.Y., Cheng, R., Ford, D., Zimmermann, T.: Investigating and Designing for Trust in AI-powered Code Generation Tools. arXiv preprint (2023). <https://doi.org/10.48550/arXiv.2305.11248>
- [2] ISO/IEC: ISO/IEC 25010:2011 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models. International Organization for Standardization (2011)
- [3] Pombo, N.: Quality Attributes and Measurement. Lecture notes, Qualidade de Software, Universidade da Beira Interior (2025)
- [4] Pombo, N.: Exploratory Testing. Lecture notes, Qualidade de Software, Universidade da Beira Interior (2025)